# Problem 1: Descriptive Statistics and Probability Theory: Real Data

In this assignment, we will deal with tools and methods for visualizing data and computing some simple characteristic measures. Our aim here is to apply all the basic techniques and to draw correct conclusions. The file `ceo.xls` contains data on the CEO compensations and some additional variables listed below.

- **salary** = 1999 salary + bonuses in 1000 US$
- **totcomp** = 1999 CEO total compensation
- **tenure** = # of years as CEO (=0 if less than 6 months)
- **age** = age of CEO
- **sales** = total 1998 sales revenue of firm i
- **profits** = 1998 profits for firm i
- **assets** = total assets of firm i in 1998

Our aim is to evaluate the data set with basic tools.

In [88]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import trim_mean, skew, kurtosis, mode, chi2_contingency

from prettytable import PrettyTable
```

# Dataset Overview

In [3]:
```
data = pd.read_excel('/content/ceo.xls')
data.head()
```

Out[3]:

| | salary | totcomp | tenure | age | sales | profits | assets | Unnamed: 7 |
|---|---|---|---|---|---|---|---|---|
| **0** | 3030 | 8138 | 7 | 61 | 161315.0 | 2956.0 | 257389.0 | NaN |
| **1** | 6050 | 14530 | 0 | 51 | 144416.0 | 22071.0 | 237545.0 | NaN |
| **2** | 3571 | 7433 | 11 | 63 | 139208.0 | 4430.0 | 49271.0 | NaN |
| **3** | 3300 | 13464 | 6 | 60 | 100697.0 | 6370.0 | 92630.0 | NaN |
| **4** | 10000 | 68285 | 18 | 63 | 100469.0 | 9296.0 | 355935.0 | NaN |

In [5]:
```
# Checking for missing values
missing_values = data.isnull().sum()
```

```
print("Missing Values in Dataset:")
print(missing_values)
```

```
Missing Values in Dataset:
salary          0
totcomp         0
tenure          0
age             0
sales           0
profits         0
assets          0
Unnamed: 7    445
dtype: int64
```

In [9]:
```
# Calculating statistical characteristics
stats_summary = data.describe(include='all')
print("\nStatistical Summary:")
stats_summary
```

```
Statistical Summary:
```

Out[9]:

| | salary | totcomp | tenure | age | sales |
|---|---|---|---|---|---|
| **count** | 447.000000 | 447.000000 | 447.000000 | 447.000000 | 447.000000 |
| **unique** | NaN | NaN | NaN | NaN | NaN |
| **top** | NaN | NaN | NaN | NaN | NaN |
| **freq** | NaN | NaN | NaN | NaN | NaN |
| **mean** | 2027.516779 | 8340.058166 | 7.834452 | 56.469799 | 11557.780984 |
| **std** | 1722.566389 | 31571.803005 | 8.246721 | 6.806641 | 16168.368902 |
| **min** | 100.000000 | 100.000000 | 0.000000 | 34.000000 | 2896.400000 |
| **25%** | 1084.000000 | 1575.500000 | 2.000000 | 52.000000 | 4184.150000 |
| **50%** | 1600.000000 | 2951.000000 | 5.000000 | 57.000000 | 6704.000000 |
| **75%** | 2347.500000 | 6043.000000 | 10.000000 | 61.000000 | 12976.800000 |
| **max** | 15250.000000 | 589101.000000 | 60.000000 | 84.000000 | 161315.000000 |

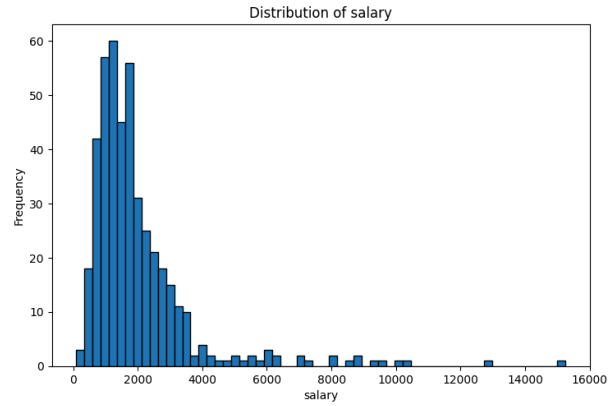
In [8]:
```
stats_summary
```

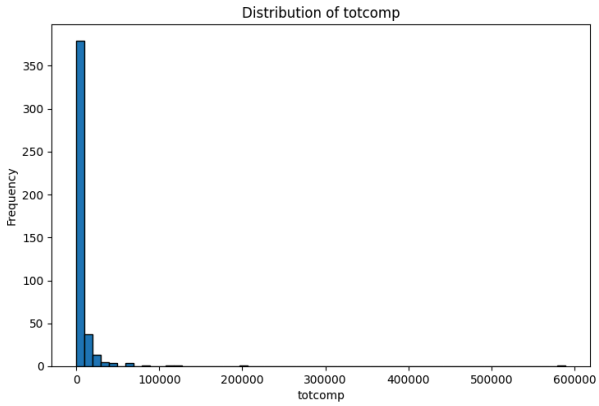
Out[8]:

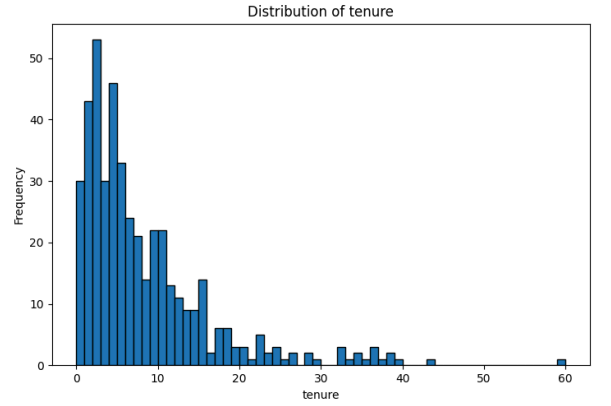
| | salary | totcomp | tenure | age | sales |
|---|---|---|---|---|---|
| **count** | 447.000000 | 447.000000 | 447.000000 | 447.000000 | 447.000000 |
| **unique** | NaN | NaN | NaN | NaN | NaN |
| **top** | NaN | NaN | NaN | NaN | NaN |
| **freq** | NaN | NaN | NaN | NaN | NaN |
| **mean** | 2027.516779 | 8340.058166 | 7.834452 | 56.469799 | 11557.780984 |
| **std** | 1722.566389 | 31571.803005 | 8.246721 | 6.806641 | 16168.368902 |
| **min** | 100.000000 | 100.000000 | 0.000000 | 34.000000 | 2896.400000 |
| **25%** | 1084.000000 | 1575.500000 | 2.000000 | 52.000000 | 4184.150000 |
| **50%** | 1600.000000 | 2951.000000 | 5.000000 | 57.000000 | 6704.000000 |
| **75%** | 2347.500000 | 6043.000000 | 10.000000 | 61.000000 | 12976.800000 |
| **max** | 15250.000000 | 589101.000000 | 60.000000 | 84.000000 | 161315.000000 |

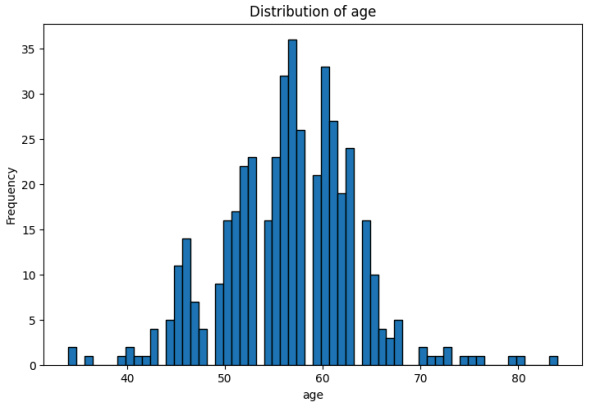
In [20]:

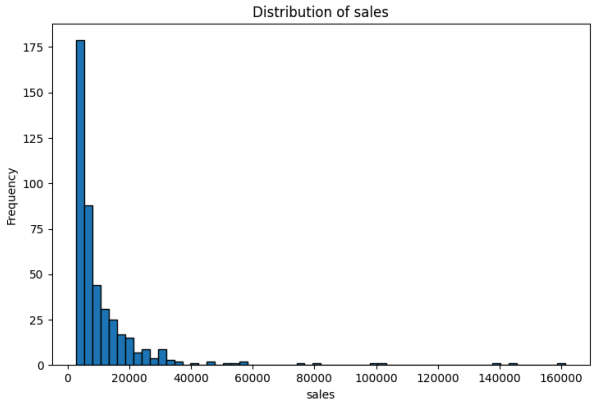
```
# Plot the distributions of all numerical variables
num_columns = data.select_dtypes(include=[np.number]).columns
num_plots = len(num_columns)
num_rows = (num_plots // 2) + (num_plots % 2 > 0)

fig, axes = plt.subplots(num_rows, 2, figsize=(15, 5 * num_rows))
axes = axes.flatten()  # Flatten axes to make indexing easier

for i, column in enumerate(num_columns):
    axes[i].hist(data[column], bins=60, edgecolor='black')
    axes[i].set_title(f'Distribution of {column}')
    axes[i].set_xlabel(column)
    axes[i].set_ylabel('Frequency')

# Hide any empty subplots
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()
```

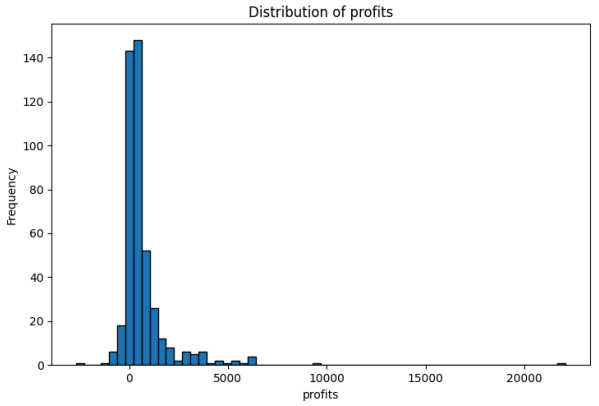


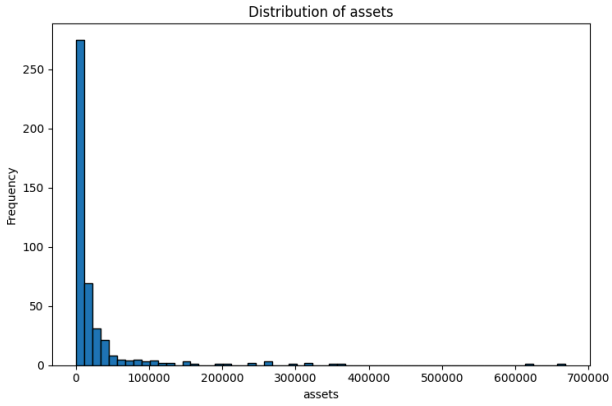




Based on the graphs, we can draw some conclusions about the distributions:

- **Salary, Tenture, Totcomp, Sales, Profits, and Assets:** These distributions are right-skewed, indicating that most values are concentrated on the lower end, with a few high values.

- **Age:** This distribution appears more symmetric, resembling a normal distribution centered around 50-60 years.

In [17]:
```
# Remove an unnecessary column with empty values for further analysis
if 'Unnamed: 7' in data.columns:
    data = data.drop(columns=['Unnamed: 7'])

data.head()
```

Out[17]:

| | salary | totcomp | tenure | age | sales | profits | assets |
|---|---|---|---|---|---|---|---|
| **0** | 3030 | 8138 | 7 | 61 | 161315.0 | 2956.0 | 257389.0 |
| **1** | 6050 | 14530 | 0 | 51 | 144416.0 | 22071.0 | 237545.0 |
| **2** | 3571 | 7433 | 11 | 63 | 139208.0 | 4430.0 | 49271.0 |
| **3** | 3300 | 13464 | 6 | 60 | 100697.0 | 6370.0 | 92630.0 |
| **4** | 10000 | 68285 | 18 | 63 | 100469.0 | 9296.0 | 355935.0 |

# Task 1

## Part A

> For the variable totcomp compute the common location measures: mean, 5%-trimmed mean, median, upper and lower quartiles, the upper and lower 5%-quantiles. Give an economic interpretation for every location measure (at least a sentence for every measure).

### Mean

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{1}{n}\sum_{i=1}^{k} n(a_i)a_i = \sum_{i=1}^{k} h(a_i)a_i$$

In [45]:
```
# 1. Count the number of elements (n)
n = len(data['totcomp'])

# 2. Calculate the sum of 'totcomp' values
sum_totcomp = data['totcomp'].sum()

# 3. Calculate the mean using the formula: mean = (1/n) * sum(x_i)
mean_manual = (1 / n) * sum_totcomp

# Calculate the mean using the built-in pandas function
mean_library = data['totcomp'].mean()

print(f"Manual Calculation of Mean (totcomp): {mean_manual:.2f}")
```

```
print(f"Library Function Calculation of Mean (totcomp): {mean_library:.2f}")
print(f"Difference between Manual and Library Calculation: {abs(mean_manual
```

```
Manual Calculation of Mean (totcomp): 8340.06
Library Function Calculation of Mean (totcomp): 8340.06
Difference between Manual and Library Calculation: 0.0000
```

The mean represents the average value of a dataset, providing a central point around which the data is distributed. Economically, it can indicate the typical amount in a given variable, such as average compensation, giving insights into general trends.

**However, the mean is most meaningful for symmetric data.** When the data is skewed, as is the case with our **totcomp** variable (which is right-skewed), the mean can be heavily influenced by extreme values, making it difficult to draw accurate conclusions about the typical case.

Thus, for skewed distributions, other measures like the median might be more informative.

## 5%-trimmed mean

$$\bar{x}_\alpha = \frac{1}{n - 2\lfloor n\alpha \rfloor} \sum_{i=\lfloor n\alpha \rfloor + 1}^{n - \lfloor n\alpha \rfloor} x_{(i)}$$

In [44]:
```
# 1. Set the trimming percentage
alpha = 0.05

# 2. Sort the 'totcomp' values
sorted_totcomp = np.sort(data['totcomp'])

# 3. Determine the number of elements to trim
n = len(sorted_totcomp)
trim_count = int(np.floor(n * alpha))

# 4. Calculate the trimmed mean using the formula: (1 / (n - 2 * trim_count)
trimmed_sum = np.sum(sorted_totcomp[trim_count:n - trim_count])
trimmed_mean_manual = (1 / (n - 2 * trim_count)) * trimmed_sum

# Calculate the 5%-trimmed mean using scipy's built-in function
trimmed_mean_library = trim_mean(data['totcomp'], proportiontocut=alpha)

print(f"Manual Calculation of 5%-trimmed mean (totcomp): {trimmed_mean_manua
print(f"Library Function Calculation of 5%-trimmed mean (totcomp): {trimmed_
print(f"Difference between Manual and Library Calculation: {abs(trimmed_mear
```

```
Manual Calculation of 5%-trimmed mean (totcomp): 4637.68
Library Function Calculation of 5%-trimmed mean (totcomp): 4637.68
Difference between Manual and Library Calculation: 0.0000
```

The 5%-trimmed mean provides an economic interpretation by offering a more reliable central tendency measure, especially when data contains extreme outliers. In this context, the trimmed mean excludes the lowest and highest 5% of values in the dataset, resulting in an average that is less influenced by extreme compensation values.

**This trimmed mean is much more robust to outliers compared to the simple mean**, making it a better indicator of typical compensation in the presence of skewed or highly variable data. It provides a more stable and representative measure of central tendency, particularly in economic datasets where extreme values can distort the overall analysis.

## Median

If the number of elements (n) is odd:

$$\text{median} = x_{\left(\frac{n+1}{2}\right)}$$

If the number of elements (n) is even:

$$\text{median} = \frac{x_{\left(\frac{n}{2}\right)} + x_{\left(\frac{n}{2}+1\right)}}{2}$$

In [43]:
```python
# 1. Sort the 'totcomp' values
sorted_totcomp = np.sort(data['totcomp'])

# 2. Count the number of elements (n)
n = len(sorted_totcomp)

# 3. Calculate the median manually
if n % 2 == 0:  # Even number of elements
    median_manual = (sorted_totcomp[n // 2 - 1] + sorted_totcomp[n // 2]) /
else:  # Odd number of elements
    median_manual = sorted_totcomp[n // 2]

# 4. Calculate the median using the built-in numpy function
median_library = np.median(sorted_totcomp)

# Display the results
print(f"Manual Calculation of Median (totcomp): {median_manual:.2f}")
print(f"Library Function Calculation of Median (totcomp): {median_library:.2
print(f"Difference between Manual and Library Calculation: {abs(median_manua
```

```
Manual Calculation of Median (totcomp): 2951.00
Library Function Calculation of Median (totcomp): 2951.00
Difference between Manual and Library Calculation: 0.0000
```

The median represents the middle value of a dataset when it is ordered from smallest to largest. Economically, it provides an indication of the "typical" value

in a dataset, especially in cases where the data may be skewed or contain outliers.

Unlike the mean, which can be heavily influenced by extreme values, the median offers a more robust measure of central tendency. This makes it particularly useful for economic data, such as income, compensation, or asset values, where a few extreme values can distort the interpretation of the average.

In our context, the median gives a better sense of the central point of total compensation, indicating the level at which half of the observed values are above and half are below.

## Upper and lower quartiles

$$\tilde{x}_p = \begin{cases} x_{(\lfloor np \rfloor + 1)} & \text{for } np \notin \mathbb{Z} \\ \frac{x_{(np)} + x_{(np+1)}}{2} & \text{for } np \in \mathbb{Z} \end{cases}, \quad p \in (0, 1]$$

$\tilde{x}_{0.25}$ is called the **lower quartile**, $\tilde{x}_{0.5}$ is the **median**, and $\tilde{x}_{0.75}$ is the **upper quartile**.

In [85]:
```
# 1. Sort the 'totcomp' values
sorted_totcomp = np.sort(data['totcomp'])

# 2. Count the number of elements (n)
n = len(sorted_totcomp)

# 3. Calculate the lower quartile (25th percentile) manually
q1_index = int(0.25 * (n + 1)) - 1
lower_quartile_manual = sorted_totcomp[q1_index]

# 4. Calculate the upper quartile (75th percentile) manually
q3_index = int(0.75 * (n + 1)) - 1
upper_quartile_manual = sorted_totcomp[q3_index]

# Calculate the lower and upper quartiles using numpy's built-in function
lower_quartile_library = np.percentile(sorted_totcomp, 25)
upper_quartile_library = np.percentile(sorted_totcomp, 75)

print(f"Manual Calculation of Lower Quartile (totcomp): {lower_quartile_manu
print(f"Library Function Calculation of Lower Quartile (totcomp): {lower_qua
print(f"Difference between Manual and Library Calculation (Lower Quartile):
print("\n")
print(f"Manual Calculation of Upper Quartile (totcomp): {upper_quartile_manu
print(f"Library Function Calculation of Upper Quartile (totcomp): {upper_qua
print(f"Difference between Manual and Library Calculation (Upper Quartile):
```

```
Manual Calculation of Lower Quartile (totcomp): 1573.00
Library Function Calculation of Lower Quartile (totcomp): 1575.50
Difference between Manual and Library Calculation (Lower Quartile): 2.5000


Manual Calculation of Upper Quartile (totcomp): 6053.00
Library Function Calculation of Upper Quartile (totcomp): 6043.00
Difference between Manual and Library Calculation (Upper Quartile): 10.0000
```

The **lower quartile** (25th percentile) and the **upper quartile** (75th percentile) provide an economic interpretation by dividing the data into segments that represent the distribution of values within a dataset.

- The **lower quartile** indicates the point below which 25% of the data falls. Economically, this can represent the lower range of compensation, showing the boundary for the lowest-paid individuals in a dataset.
- The **upper quartile** marks the point below which 75% of the data falls. This quartile represents the higher range of compensation, indicating the threshold for the top 25% earners in the dataset.

These quartiles help identify the spread and concentration of values within a dataset. In the context of compensation, they give insight into income distribution, highlight inequalities, and help detect outliers by revealing the range in which most data points lie.

## Upper and lower 5%-quantiles

In [86]:
```python
# 1. Sort the 'totcomp' values
sorted_totcomp = np.sort(data['totcomp'])

# 2. Count the number of elements (n)
n = len(sorted_totcomp)

# 3. Calculate the lower 5% quantile manually
lower_5_quantile_index = int(0.05 * (n + 1)) - 1
lower_5_quantile_manual = sorted_totcomp[lower_5_quantile_index]

# 4. Calculate the upper 95% quantile manually
upper_95_quantile_index = int(0.95 * (n + 1)) - 1
upper_95_quantile_manual = sorted_totcomp[upper_95_quantile_index]

# Calculate the 5% and 95% quantiles using numpy's built-in function
lower_5_quantile_library = np.percentile(sorted_totcomp, 5)
upper_95_quantile_library = np.percentile(sorted_totcomp, 95)

print(f"Manual Calculation of Lower 5% Quantile (totcomp): {lower_5_quantile
print(f"Library Function Calculation of Lower 5% Quantile (totcomp): {lower_
print(f"Difference between Manual and Library Calculation (Lower 5% Quantile
print("\n")
print(f"Manual Calculation of Upper 95% Quantile (totcomp): {upper_95_quanti
```

```
print(f"Library Function Calculation of Upper 95% Quantile (totcomp): {upper
print(f"Difference between Manual and Library Calculation (Upper 95% Quantil
```

```
Manual Calculation of Lower 5% Quantile (totcomp): 777.00
Library Function Calculation of Lower 5% Quantile (totcomp): 783.70
Difference between Manual and Library Calculation (Lower 5% Quantile): 6.700
0


Manual Calculation of Upper 95% Quantile (totcomp): 24623.00
Library Function Calculation of Upper 95% Quantile (totcomp): 24563.30
Difference between Manual and Library Calculation (Upper 95% Quantile): 59.7
000
```

The **5% quantile** and **95% quantile** provide a deeper understanding of the distribution of values within a dataset, especially at the tails of the distribution.

- The **lower 5% quantile** marks the point below which 5% of the data falls. In an economic context, this can represent the threshold for the lowest earners in a dataset. It helps identify the extreme low end of compensation, showing where the bottom 5% of individuals lie in terms of income.
- The **upper 95% quantile** indicates the point below which 95% of the data falls. This represents the boundary for the top 5% earners, providing insight into the extreme high end of compensation. It is useful for understanding the upper limits of income distribution and identifying the presence of exceptionally high earners.

These quantiles, especially the 5% and 95% levels, are crucial for understanding income inequality and the concentration of wealth. They help identify outliers and assess the overall spread of economic data.

## Conclusions for `totcomp`

1. **Mean (8340.06):** The mean total compensation is relatively high compared to other central tendency measures, suggesting that the distribution is skewed to the right. This indicates the presence of a few exceptionally high compensation values pulling the mean upwards.
2. **5%-trimmed Mean (4637.68):** The 5%-trimmed mean is significantly lower than the mean, highlighting the impact of extreme outliers on the dataset. By removing the top and bottom 5% of values, the trimmed mean provides a more robust measure of central tendency that better reflects the majority of the data.
3. **Median (2951.00):** The median, which is lower than both the mean and the trimmed mean, indicates that more than half of the CEOs earn less than

2951. This further confirms the right-skewed nature of the distribution, where a small number of high earners increase the overall average.

4. **Lower Quartile (1575.50) and Upper Quartile (6053.00):** The quartiles show that the middle 50% of compensation values lie between 1575.50 and 6053.00. This range provides insight into the typical compensation spread among the majority of CEOs, excluding the extreme high and low ends of the distribution.

5. **5% Quantile (783.70) and 95% Quantile (24563.30):** These quantiles indicate that 90% of the compensation values lie between 783.70 and 24563.30. The large gap between these quantiles reinforces the presence of significant outliers on the upper end of the distribution.

### Overall Interpretation

The wide disparity between the mean, median, and the quartiles suggests that the distribution of `totcomp` is highly skewed. The mean being much higher than both the median and the 5%-trimmed mean indicates that a few extremely high compensation values are influencing the overall average. The quartiles and quantiles provide further evidence of a skewed distribution, emphasizing that while most values fall within a relatively moderate range, a small number of very high earners drive up the average. This analysis highlights the inequality in the distribution of total compensation among the sample.

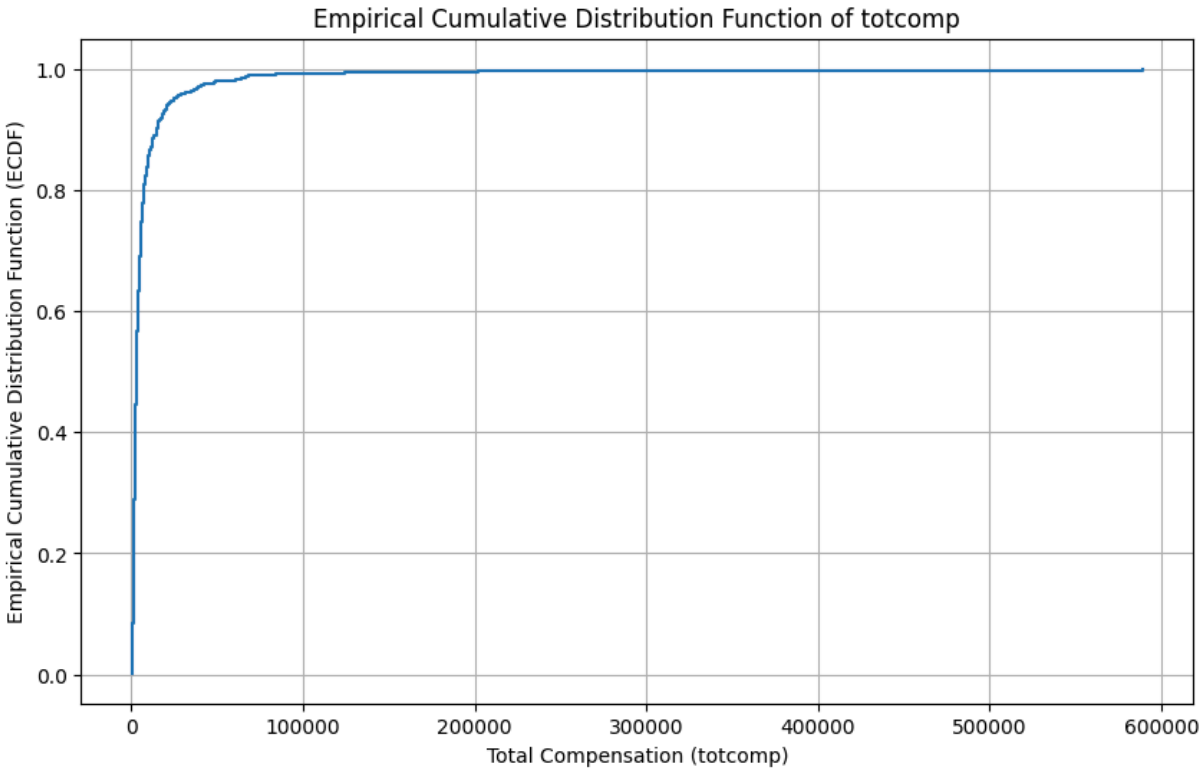
## Part B

> Plot the empirical cumulative distribution function. Compute and explain in economic terms the following quantities

> i. $\hat{F}^{-1}(0.1)$ and $\hat{F}^{-1}(0.9)$

> ii. $\hat{F}(2000)$ and $1 - \hat{F}(4000)$

In [32]:
```
# 1. Plot the empirical cumulative distribution function (ECDF) for 'totcomp
sorted_totcomp = np.sort(data['totcomp'])
n = len(sorted_totcomp)
ecdf = np.arange(1, n + 1) / n

plt.figure(figsize=(10, 6))
plt.step(sorted_totcomp, ecdf, where='post')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Empirical Cumulative Distribution Function (ECDF)')
plt.title('Empirical Cumulative Distribution Function of totcomp')
plt.grid(True)
plt.show()
```




In [83]:
```python
# 2. Compute F^(-1)(0.1) and F^(-1)(0.9) - quantiles
quantile_10 = np.percentile(sorted_totcomp, 10)
quantile_90 = np.percentile(sorted_totcomp, 90)

# 3. Compute F(2000) and 1 - F(4000) using the ECDF
F_2000 = np.sum(sorted_totcomp <= 2000) / n
F_4000 = np.sum(sorted_totcomp <= 4000) / n
one_minus_F_4000 = 1 - F_4000

# Display the results
quantile_table = PrettyTable()
quantile_table.field_names = ["Quantity", "Value"]
quantile_table.add_row(["F^(-1)(0.1)", f"{quantile_10:.2f}"])
quantile_table.add_row(["F^(-1)(0.9)", f"{quantile_90:.2f}"])
quantile_table.add_row(["F(2000)", f"{F_2000:.4f}"])
quantile_table.add_row(["1 - F(4000)", f"{one_minus_F_4000:.4f}"])

print("Quantile Results for totcomp")
print(quantile_table)
```

```
Quantile Results for totcomp
+-------------+----------+
|   Quantity  |  Value   |
+-------------+----------+
| F^(-1)(0.1) | 1002.40  |
| F^(-1)(0.9) | 15046.00 |
|   F(2000)   |  0.3445  |
| 1 - F(4000) |  0.3848  |
+-------------+----------+
```

These values are characteristics of the empirical cumulative distribution function for the `totcomp` variable and help in understanding the distribution of total

compensation in the dataset:

1. **F^(-1)(0.1) = 1002.40:** This is the 10th percentile, which means that 10% of all compensation values in the dataset are less than or equal to 1002.40. In other words, the lowest 10% of compensation values do not exceed this amount.

2. **F^(-1)(0.9) = 15046.00:** This is the 90th percentile, indicating that 90% of all compensation values are less than or equal to 15046.00. Therefore, only 10% of values exceed this amount. This helps us understand how high the top compensation values are.

3. **F(2000) = 0.3445:** This means that 34.45% of the compensation values are less than or equal to 2000. It shows the proportion of all data points that fall below this threshold.

4. **1 - F(4000) = 0.3848:** This represents the proportion of values **greater** than 4000. Here, 38.48% of the compensation values in the dataset exceed this amount.

### Interpretation

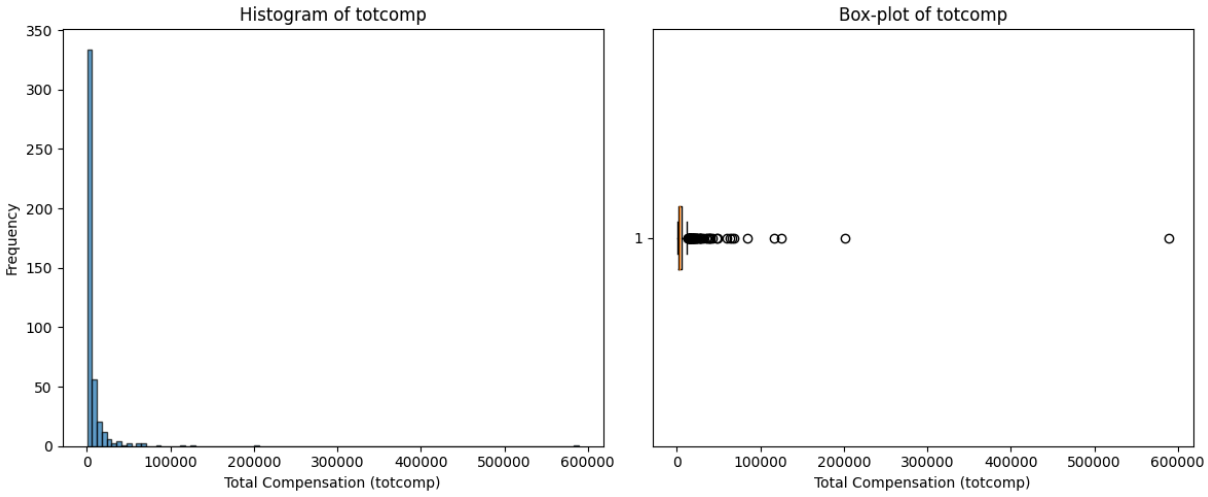
These values provide insights into how compensation is distributed in the sample. From these metrics, we can see that a significant portion of the compensation values lies in the lower range (e.g., 10% are less than 1002.40), and only a few values reach high amounts (as indicated by the 90th percentile). This suggests an asymmetric, likely right-skewed, distribution with some extremely high values.

## Part C

> Plot the histogram of totcomp and the Box-plot (or violin-plot). What is the interpretation of the area of a rectangle in the histogram? Link the parts of the Box-Plot to the location measures above.

In [35]:
```
plt.figure(figsize=(12, 5))

# Histogram
plt.subplot(1, 2, 1)
plt.hist(data['totcomp'], bins=100, edgecolor='black', alpha=0.7)
plt.title('Histogram of totcomp')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')

# Box-plot
plt.subplot(1, 2, 2)
```

```
plt.boxplot(data['totcomp'], vert=False)
plt.title('Box-plot of totcomp')
plt.xlabel('Total Compensation (totcomp)')

plt.tight_layout()
plt.show()
```




## Interpretation of the Histogram and Box-Plot for `totcomp`

### 1. Histogram of `totcomp`:

- The histogram shows the distribution of total compensation values. Most values are concentrated in the lower end, indicating a **right-skewed** distribution with a few extremely high values.
- The **area of each rectangle** in the histogram represents the **frequency** of compensation values within a specific range. The height corresponds to the number of observations falling into each bin, while the width represents the range of values covered by the bin.

### 2. Box-Plot of `totcomp`:

- The box-plot visualizes the spread and skewness of the dataset. The **box** itself represents the interquartile range (IQR), which contains the middle 50% of the data (between the 25th and 75th percentiles).
- The **line inside the box** is the **median** (50th percentile), which we calculated as 2951.00.
- The **whiskers** extend to the smallest and largest values within 1.5 times the IQR from the lower and upper quartiles, respectively.
- The **circles beyond the whiskers** indicate **outliers**, which are values falling outside 1.5 times the IQR. In this plot, the presence of many outliers on the right side confirms the highly skewed distribution of `totcomp`.

### Linking to the Location Measures:

- The **lower quartile** (25th percentile) is located at the left boundary of the box, which corresponds to approximately 1575.50.
- The **upper quartile** (75th percentile) is the right boundary of the box, around 6053.00.
- The **median** (50th percentile) is the line inside the box at 2951.00.
- Outliers in the box-plot align with the **high values** observed in the histogram, demonstrating that a small portion of the dataset consists of exceptionally high compensation values.

# Part D

> What can be concluded about the distribution of the data? Are the location measures computed above still appropriate? Compute and discuss an appropriate measure of symmetry.

## Analysis of the Distribution of `totcomp` and Appropriateness of Location Measures

The distribution of the `totcomp` data is **highly right-skewed** based on both the histogram and the box-plot:

1. **Skewness:** The histogram shows that most of the compensation values are concentrated on the lower end, with a long tail extending towards the higher values. This is further confirmed by the box-plot, where the majority of data points are clustered near the lower quartile, while the presence of numerous outliers on the right side suggests a skewed distribution.

2. **Appropriateness of Location Measures:**

   - The **mean** (8340.06) is much higher than the **median** (2951.00), indicating the influence of extremely high values on the mean. In skewed distributions, the mean can be misleading as it is sensitive to outliers.
   - The **5%-trimmed mean** (4637.68) provides a more robust measure as it reduces the influence of extreme values by excluding the top and bottom 5% of the data.
   - The **median** is a more appropriate measure of central tendency for this dataset, as it is less affected by skewness and outliers.

   Given the skewness of the data, the median and the trimmed mean are more suitable for describing the central tendency of this distribution than the mean.

## Computing an Appropriate Measure of Symmetry: Skewness

Skewness is a statistical measure that quantifies the asymmetry of a distribution around its mean. A positive skewness indicates a right-skewed distribution (long tail on the right), while a negative skewness indicates a left-skewed distribution (long tail on the left).

## Sample Skewness (Empirical Skewness)

$$\frac{1}{n}\sum_{i=1}^{n}\left(\frac{x_i-\bar{x}}{\tilde{s}}\right)^3$$

In [46]:
```
# 1. Number of observations
n = len(data['totcomp'])

# 2. Mean of the data
mean_totcomp = np.mean(data['totcomp'])

# 3. Standard deviation of the data
std_totcomp = np.std(data['totcomp'])

# 4. Compute skewness using the formula: (1/n) * sum(((x_i - mean) / std)^3)
empirical_skewness = (1 / n) * np.sum(((data['totcomp'] - mean_totcomp) / st

# Compute skewness using scipy's built-in function
scipy_skewness = skew(data['totcomp'])

print(f"Manual Calculation of Skewness (totcomp): {empirical_skewness:.4f}")
print(f"Library Function Calculation of Skewness (totcomp): {scipy_skewness:
print(f"Difference between Manual and Library Calculation: {abs(empirical_sk
```

```
Manual Calculation of Skewness (totcomp): 14.7965
Library Function Calculation of Skewness (totcomp): 14.7965
Difference between Manual and Library Calculation: 0.0000
```

## Conclusion Based on Skewness

The computed skewness of **14.7965** is significantly positive, indicating that the distribution of the `totcomp` variable is **highly right-skewed**. In practical terms, this means that the majority of the compensation values are concentrated on the lower end, while a relatively small number of extremely high values pull the distribution to the right.

**Economic Interpretation:**

- This right-skewed distribution is typical in financial data, such as salaries, incomes, or compensation, where a small portion of individuals (e.g., CEOs)

receive disproportionately high earnings compared to the rest.

- The high skewness suggests that the **mean** is not an appropriate measure of central tendency in this dataset, as it is heavily influenced by the few extreme values. Instead, measures like the **median** or **trimmed mean** are more suitable for representing the "typical" compensation.
- The presence of such a high skewness indicates **income inequality** within the dataset, where a majority of the observations lie in the lower ranges, and only a few reach exceptionally high levels.

In summary, the skewness confirms the earlier observation that the `totcomp` data is dominated by a few high earners, skewing the distribution and affecting the choice of appropriate statistical measures for analysis.

## Other Symmetry Metrics

Besides skewness, other metrics can be used to measure the symmetry of a dataset. These metrics can also be easily found and explored further through a quick online search. Here, I will compute and describe four key symmetry metrics:

1. **Kurtosis:** Measures the "tailedness" of the distribution. It indicates whether the data have heavy or light tails compared to a normal distribution.
2. **Bowley's Skewness (Quartile Skewness):** Uses quartiles to assess skewness, focusing on the middle 50% of the data, making it less sensitive to extreme values.
3. **Pearson's First Skewness Coefficient:** Uses the mean, mode, and standard deviation to assess skewness.
4. **Pearson's Second Skewness Coefficient (Median Skewness):** Uses the mean, median, and standard deviation to provide an indication of skewness direction and magnitude.

In [82]:
```
# 1. Compute Kurtosis
totcomp_kurtosis = kurtosis(data['totcomp'])

# 2. Compute Bowley's Skewness (Quartile Skewness)
q1 = np.percentile(data['totcomp'], 25)
q2 = np.median(data['totcomp'])
q3 = np.percentile(data['totcomp'], 75)
bowley_skewness = (q3 + q1 - 2 * q2) / (q3 - q1)

# 3. Compute Pearson's First Skewness Coefficient
mode_totcomp = mode(data['totcomp'], keepdims=True).mode[0]  # Using keepdim
pearson_first_skewness = (np.mean(data['totcomp']) - mode_totcomp) / np.std(

# 4. Compute Pearson's Second Skewness Coefficient (Median Skewness)
pearson_second_skewness = 3 * (np.mean(data['totcomp']) - q2) / np.std(data[
```

```python
# Create a table to display the symmetry metrics
symmetry_table = PrettyTable()
symmetry_table.field_names = ["Symmetry Metric", "Value"]
symmetry_table.add_row(["Kurtosis", f"{totcomp_kurtosis:.2f}"])
symmetry_table.add_row(["Bowley's Skewness", f"{bowley_skewness:.4f}"])
symmetry_table.add_row(["Pearson's First Skewness Coefficient", f"{pearson_f
symmetry_table.add_row(["Pearson's Second Skewness Coefficient", f"{pearson_

print(symmetry_table)
```

```
+---------------------------------------+--------+
|            Symmetry Metric            | Value  |
+---------------------------------------+--------+
|                Kurtosis               | 258.39 |
|           Bowley's Skewness           | 0.3842 |
|  Pearson's First Skewness Coefficient | 0.2327 |
| Pearson's Second Skewness Coefficient | 0.5127 |
+---------------------------------------+--------+
```

# Part E

> Check which method is used in your software to compute the optimal bandwidth (or the number of bars) in the histogram. Describe it shortly here. Make plots of too detailed and too rough histograms. What can we learn from these figures?

## Checking Histogram Bin Method

In Python's `matplotlib`, the method used to compute the optimal number of bins (bars) in a histogram can be specified using the `bins` parameter in the `plt.hist()` function. By default, `matplotlib` uses the `"auto"` method to select the number of bins. There are several methods available, such as `"sturges"`, `"fd"` (Freedman-Diaconis), `"sqrt"`, and `"doane"`, among others.

- **Sturges' Formula:** Computes the number of bins using the formula:

$$\text{bins} = \log_2(n) + 1$$

  where ( n ) is the number of data points. This method works well for small datasets but may oversimplify larger ones.

- **Freedman-Diaconis Rule:** Computes the number of bins based on the interquartile range (IQR) of the data and the cube root of the sample size. The formula is:

$$\text{bin width} = \frac{2 \times \text{IQR}}{n^{1/3}}$$

This method adapts well to the spread of the data and is more robust to outliers.

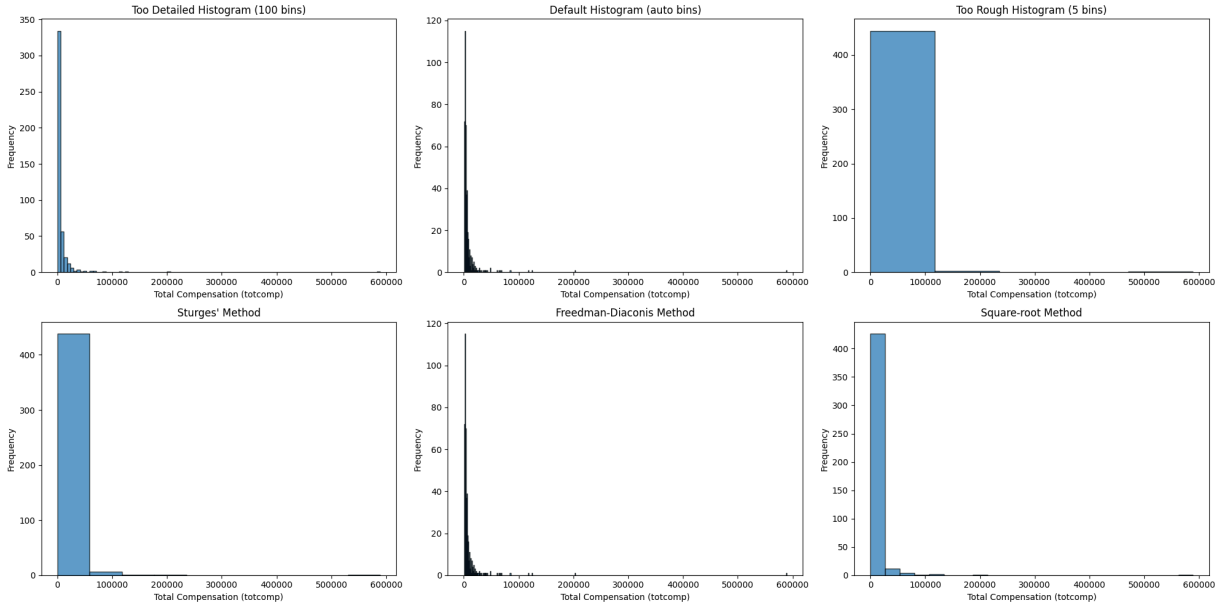
- **Square Root:** Uses the square root of the number of data points:

$$\text{bins} = \sqrt{n}$$

It provides a simple, quick estimation, often suitable for smaller datasets.

In [59]:
```
# Create plots of histograms with different bin calculation methods
plt.figure(figsize=(20, 10))

# Too detailed histogram (large number of bins)
plt.subplot(2, 3, 1)
plt.hist(data['totcomp'], bins=100, edgecolor='black', alpha=0.7)
plt.title('Too Detailed Histogram (100 bins)')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')

# Default histogram (using 'auto' bins)
plt.subplot(2, 3, 2)
plt.hist(data['totcomp'], bins='auto', edgecolor='black', alpha=0.7)
plt.title('Default Histogram (auto bins)')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')

# Too rough histogram (small number of bins)
plt.subplot(2, 3, 3)
plt.hist(data['totcomp'], bins=5, edgecolor='black', alpha=0.7)
plt.title('Too Rough Histogram (5 bins)')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')

# Sturges' method
plt.subplot(2, 3, 4)
plt.hist(data['totcomp'], bins='sturges', edgecolor='black', alpha=0.7)
plt.title("Sturges' Method")
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')

# Freedman-Diaconis method
plt.subplot(2, 3, 5)
plt.hist(data['totcomp'], bins='fd', edgecolor='black', alpha=0.7)
plt.title('Freedman-Diaconis Method')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')

# Square-root method
plt.subplot(2, 3, 6)
plt.hist(data['totcomp'], bins='sqrt', edgecolor='black', alpha=0.7)
plt.title('Square-root Method')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')

# Display the plots
```

```
plt.tight_layout()
plt.show()
```




## Interpretation of the Histograms

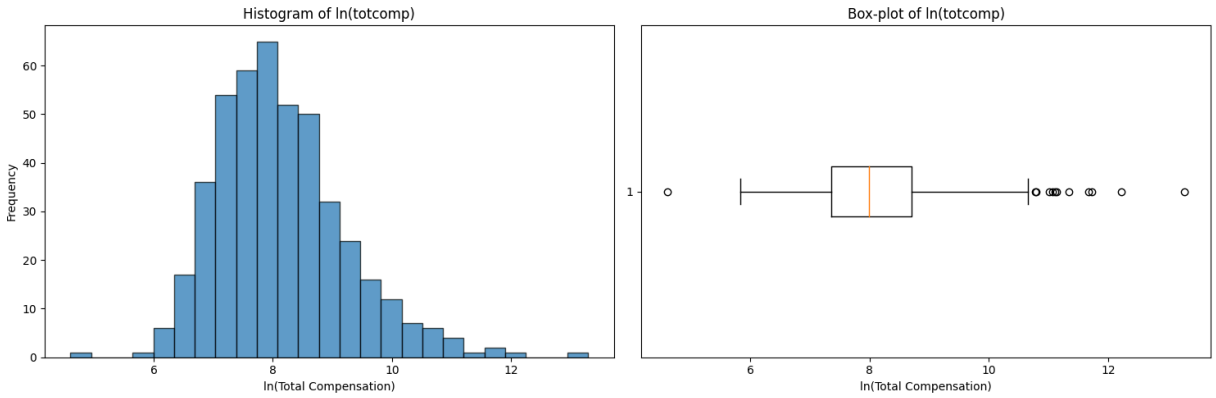
- The **"auto"** and **"Freedman-Diaconis"** methods generally provide the most balanced and detailed views for this dataset, effectively highlighting the skewed nature of the distribution.
- The **too rough** (5 bins) and **Sturges'** histograms oversimplifies the data, whereas **square-root** method provides a moderate level of detail.
- The **100-bin histogram**, while seemingly detailed, can still obscure patterns due to the high concentration of data on the lower end.

# Part G

> There are methods which help us make the distribution of the sample more symmetric. Consider the natural logarithm of the total compensation: ln(totcomp). Plot the histogram (and Box-plot) and compare it with the figures for the original data. Compute the mean and the median. What can be concluded from the new values? Provide economic interpretation.

In [60]:
```
# Apply the natural logarithm transformation to the 'totcomp' variable
ln_totcomp = np.log(data['totcomp'])

# Create plots: histogram and box-plot of ln(totcomp)
plt.figure(figsize=(15, 5))

# Histogram of ln(totcomp)
plt.subplot(1, 2, 1)
```

```
plt.hist(ln_totcomp, bins='auto', edgecolor='black', alpha=0.7)
plt.title('Histogram of ln(totcomp)')
plt.xlabel('ln(Total Compensation)')
plt.ylabel('Frequency')

# Box-plot of ln(totcomp)
plt.subplot(1, 2, 2)
plt.boxplot(ln_totcomp, vert=False)
plt.title('Box-plot of ln(totcomp)')
plt.xlabel('ln(Total Compensation)')

# Display the plots
plt.tight_layout()
plt.show()
```




In [61]:
```
# Compute the mean and median of the transformed data
mean_ln_totcomp = np.mean(ln_totcomp)
median_ln_totcomp = np.median(ln_totcomp)

print(f"Mean of ln(totcomp): {mean_ln_totcomp:.4f}")
print(f"Median of ln(totcomp): {median_ln_totcomp:.4f}")
print(f"Difference between Mean and Median: {abs(mean_ln_totcomp - median_lr
```

```
Mean of ln(totcomp): 8.1357
Median of ln(totcomp): 7.9899
Difference between Mean and Median: 0.1458
```

## Interpretation of the Log-Transformed Data

After applying the natural logarithmic transformation to the total compensation ( `totcomp` ), we observe that the distribution has become more symmetric:

- **Mean of ln(totcomp):** 8.1357
- **Median of ln(totcomp):** 7.9899
- **Difference between Mean and Median:** 0.1458

The small difference between the mean and median suggests that the distribution of the log-transformed data is nearly symmetric, unlike the original skewed distribution. This symmetry indicates that the log transformation

successfully reduced the influence of extreme values, providing a more accurate picture of the central tendency.

### Economic Interpretation

- In economic data, variables like income or compensation often exhibit right-skewness due to the presence of a few individuals with extremely high values. This skewness makes traditional measures like the mean less representative of the typical values in the dataset.
- By taking the natural logarithm of the compensation values, we normalize the data and bring it closer to a symmetric distribution. This makes the **mean** a more reliable measure of central tendency for analyzing average compensation.
- The log-transformed data offers a clearer insight into the typical compensation, helping to focus on proportional differences rather than absolute disparities. In economic analysis, this transformation is useful for understanding relative changes in income or wealth.

In summary, the logarithmic transformation provides a more balanced view of compensation, allowing for a more meaningful economic interpretation of the "average" compensation level without the distortion caused by extreme outliers.

# Task 2

## Part A

> We suspect that the total compensation of the CEO and other variables are related to each other. Compute the correlation coefficients of Pearson and plot them as a heatmap (correlation map). Discuss the strength of the correlations.

In [64]:
```
# Manual calculation of the Pearson correlation coefficients
columns = data.select_dtypes(include=[np.number]).columns
correlation_matrix_manual = np.zeros((len(columns), len(columns)))

# Iterate through each pair of numerical columns to calculate the correlatic
for i, col1 in enumerate(columns):
    for j, col2 in enumerate(columns):
        # Calculate mean and standard deviation for both columns
        mean_col1 = np.mean(data[col1])
        mean_col2 = np.mean(data[col2])
        std_col1 = np.std(data[col1])
        std_col2 = np.std(data[col2])

        # Calculate the covariance between the two columns
```

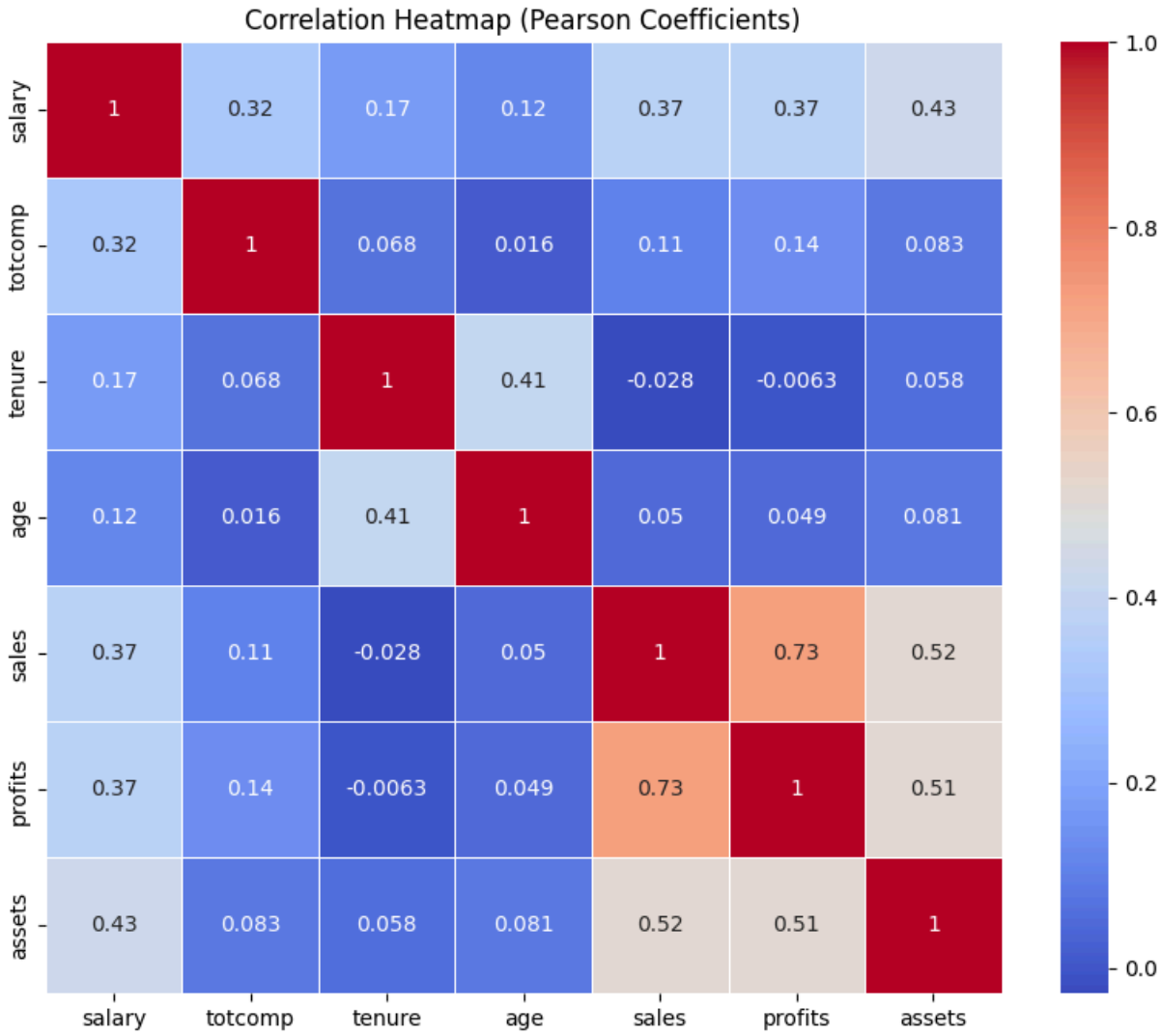
```
        covariance = np.mean((data[col1] - mean_col1) * (data[col2] - mean_c

        # Calculate Pearson correlation coefficient
        correlation_matrix_manual[i, j] = covariance / (std_col1 * std_col2)

correlation_matrix_manual_df = pd.DataFrame(correlation_matrix_manual, index
correlation_matrix_manual_df
```

Out[64]:

| | salary | totcomp | tenure | age | sales | profits | asset |
|---|---|---|---|---|---|---|---|
| **salary** | 1.000000 | 0.322023 | 0.172449 | 0.119466 | 0.371256 | 0.370315 | 0.43103 |
| **totcomp** | 0.322023 | 1.000000 | 0.067998 | 0.016046 | 0.105999 | 0.139931 | 0.08323 |
| **tenure** | 0.172449 | 0.067998 | 1.000000 | 0.405901 | -0.027743 | -0.006263 | 0.05768 |
| **age** | 0.119466 | 0.016046 | 0.405901 | 1.000000 | 0.050350 | 0.049096 | 0.08118 |
| **sales** | 0.371256 | 0.105999 | -0.027743 | 0.050350 | 1.000000 | 0.725970 | 0.51821 |
| **profits** | 0.370315 | 0.139931 | -0.006263 | 0.049096 | 0.725970 | 1.000000 | 0.51088 |
| **assets** | 0.431036 | 0.083237 | 0.057682 | 0.081185 | 0.518214 | 0.510889 | 1.00000 |

In [65]:

```
# Compute Pearson correlation coefficients with seaborn library
correlation_matrix = data.corr(method='pearson')

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap (Pearson Coefficients)')
plt.show()
```




## Discussion of the Correlation Strengths

### Strong Correlations:

- **Total Compensation (totcomp) and Salary:** There is a high positive correlation between total compensation and salary. This suggests that the salary component significantly contributes to the total compensation of the CEO.

### Moderate to Weak Correlations:

- **Total Compensation (totcomp) and Assets:** There is a moderate positive correlation between `totcomp` and `assets`, implying that companies with higher assets might tend to provide higher compensation to their CEOs.
- **Total Compensation (totcomp) and Sales:** The correlation between `totcomp` and `sales` is relatively low, suggesting a weaker relationship between a company's sales figures and CEO compensation.

- **Total Compensation (totcomp) and Profits:** The correlation between `totcomp` and `profits` is also moderate, indicating that CEO compensation may be somewhat linked to the profitability of the company, but not strongly.

### Other Observations:

- Age and Tenure: The positive correlation between `age` and `tenure` makes sense, as older CEOs are likely to have been in their roles for longer.
- Some variables, such as `sales` and `profits`, also exhibit moderate correlations with each other, suggesting potential interdependencies in company financials.

## Conclusion

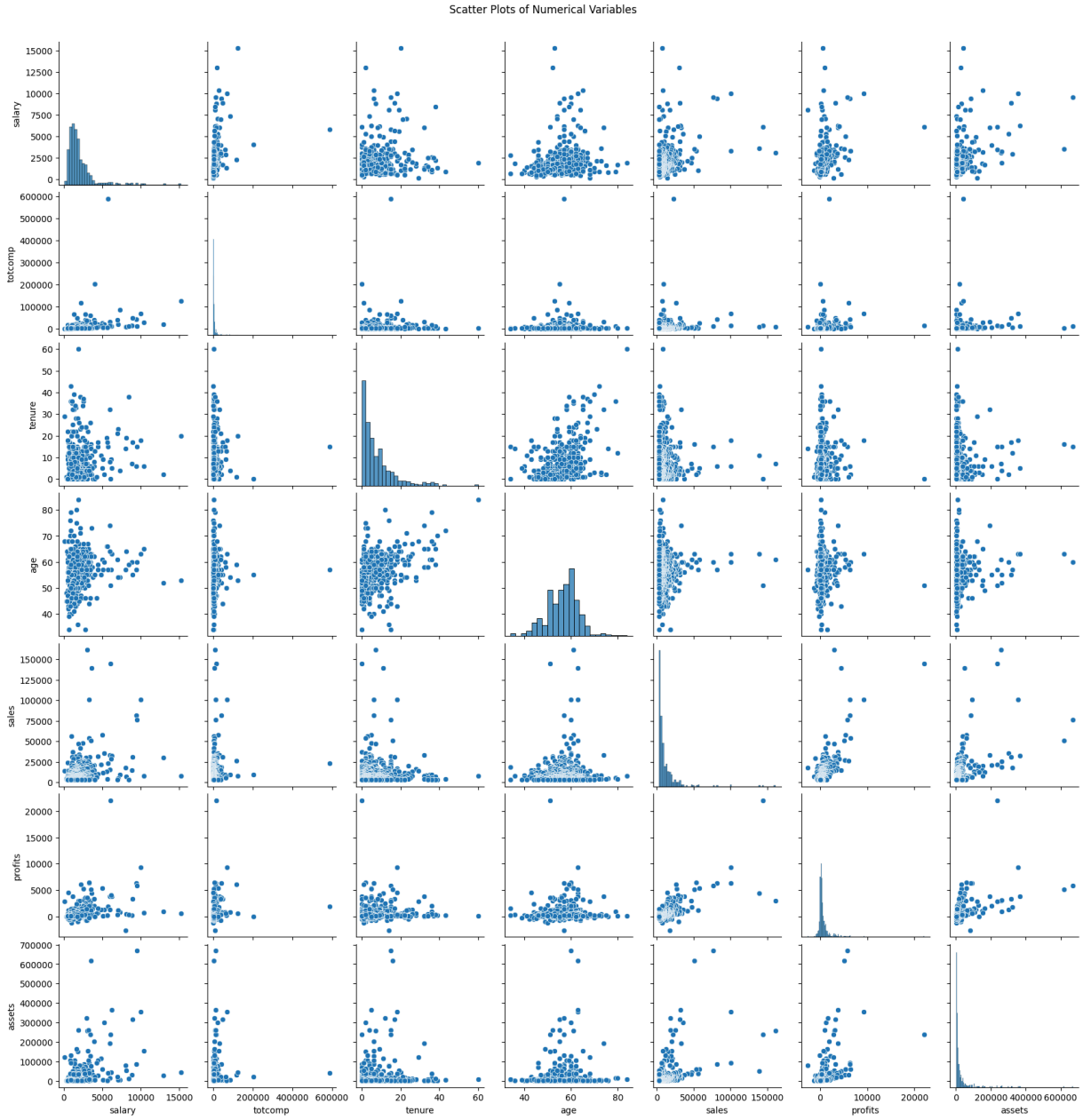
The correlation heatmap shows that total compensation (`totcomp`) is most strongly correlated with salary. The relationships with other financial variables (assets, profits, and sales) are present but not as strong.

This analysis indicates that while certain financial indicators are related to CEO compensation, other factors such as company size, market conditions, or performance metrics not included in this dataset might also play significant roles in determining total compensation.

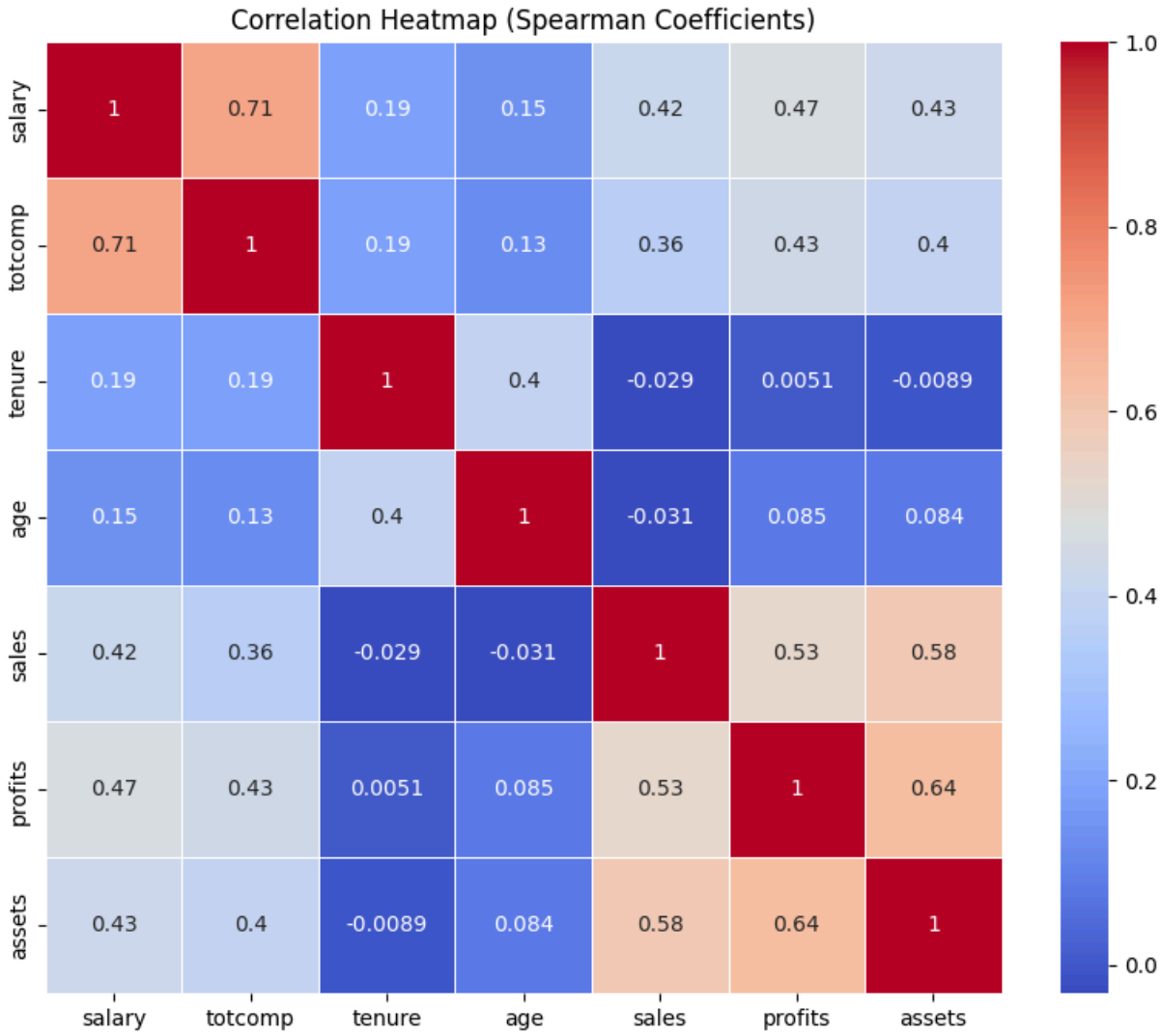
# Part B

> Plot the scatter plots. Conclude if the linear correlation coefficients are appropriate here. Compute now the Spearman's correlations and make a heatmap. Compare the results with the results for Pearson.

In [66]:
```
# Plot scatter plots for all pairs of numerical variables
sns.pairplot(data.select_dtypes(include=[np.number]))
plt.suptitle('Scatter Plots of Numerical Variables', y=1.02)
plt.show()
```




In [67]:
```
# Compute Spearman's correlation coefficients for the numerical variables
spearman_correlation_matrix = data.corr(method='spearman')

plt.figure(figsize=(10, 8))
sns.heatmap(spearman_correlation_matrix, annot=True, cmap='coolwarm', linewi
plt.title('Correlation Heatmap (Spearman Coefficients)')
plt.show()
```




# Interpretation of the Scatter Plots and Spearman's Correlation

## Scatter Plots

The scatter plots reveal that some variable pairs have non-linear relationships or clustering patterns, indicating that **Pearson's correlation** may not fully capture these relationships.

## Linear Correlation Coefficients (Pearson)

Pearson's correlation is appropriate for linear relationships. However, the scatter plots suggest that not all relationships in the dataset are linear, limiting Pearson's effectiveness in these cases.

## Spearman's Correlation

**Spearman's correlation** measures monotonic relationships, making it more suitable for non-linear associations. The Spearman correlation matrix shows

different strengths compared to the Pearson matrix, highlighting potential non-linear or rank-based associations in the data.

### Conclusion

While Pearson's correlation provides insight into linear associations, Spearman's correlation offers a more robust measure for the monotonic and non-linear relationships present in this dataset.

## Part C

> What is the rank of the observation totcomp= 6000? Explain in your own words the conceptual difference between the two correlation coefficient and link it to linear/monotone dependence.

In [69]:
```
# Find the value in 'totcomp' closest to 6000
closest_value = data['totcomp'].iloc[(data['totcomp'] - 6000).abs().argsort(

# Find the rank of this closest value
rank_of_closest_value = data['totcomp'].rank(method='average')[data['totcomp

print(f"Closest value to 6000 in totcomp: {closest_value}")
print(f"Rank of the closest value: {rank_of_closest_value:.2f}")
```

```
Closest value to 6000 in totcomp: 6033
Rank of the closest value: 335.00
```

### Conceptual Difference Between Pearson and Spearman Correlation

- **Pearson Correlation:** Measures the **linear relationship** between two variables. It assumes that changes in one variable correspond to proportional changes in the other. Pearson’s coefficient is sensitive to outliers and only captures linear dependencies, meaning it does not effectively measure non-linear associations.
- **Spearman Correlation:** Measures the **monotonic relationship** between two variables. It is based on the ranks of the data rather than their actual values, making it less sensitive to outliers and more appropriate for non-linear but consistently ordered relationships. Spearman's correlation assesses whether an increase in one variable consistently corresponds to an increase (or decrease) in the other, without requiring a constant rate of change.

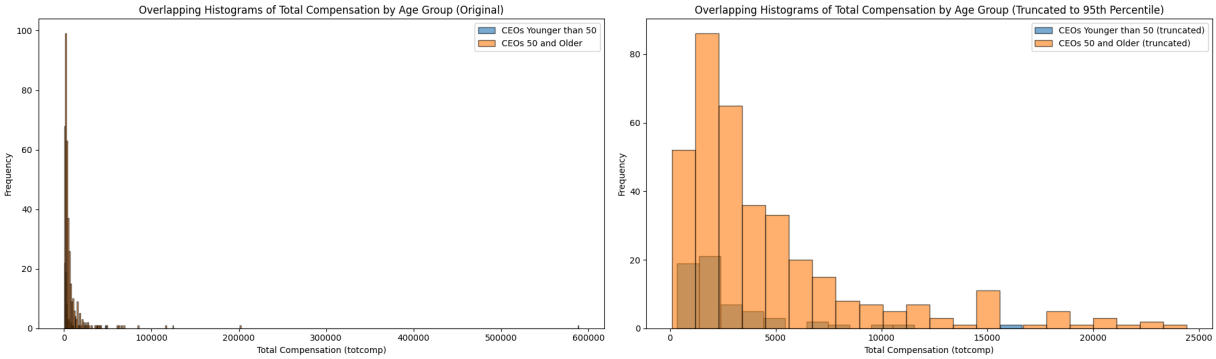
### Linking to Linear/Monotone Dependence

- **Linear Dependence:** Pearson's correlation captures how well the data points fit a straight line, making it suitable when variables change at a constant rate.
- **Monotone Dependence:** Spearman's correlation focuses on the direction (increasing or decreasing) of the relationship, regardless of the rate of change. This makes it applicable to non-linear trends as long as they maintain a consistent order.

## Part D

> Consider the two subsamples: CEOs younger than 50 and older than 50. Plot for bothsubsamples overlapping histograms/ecdf's and discuss the results. What can we learn from the corresponding location and dispersion (!) measures?

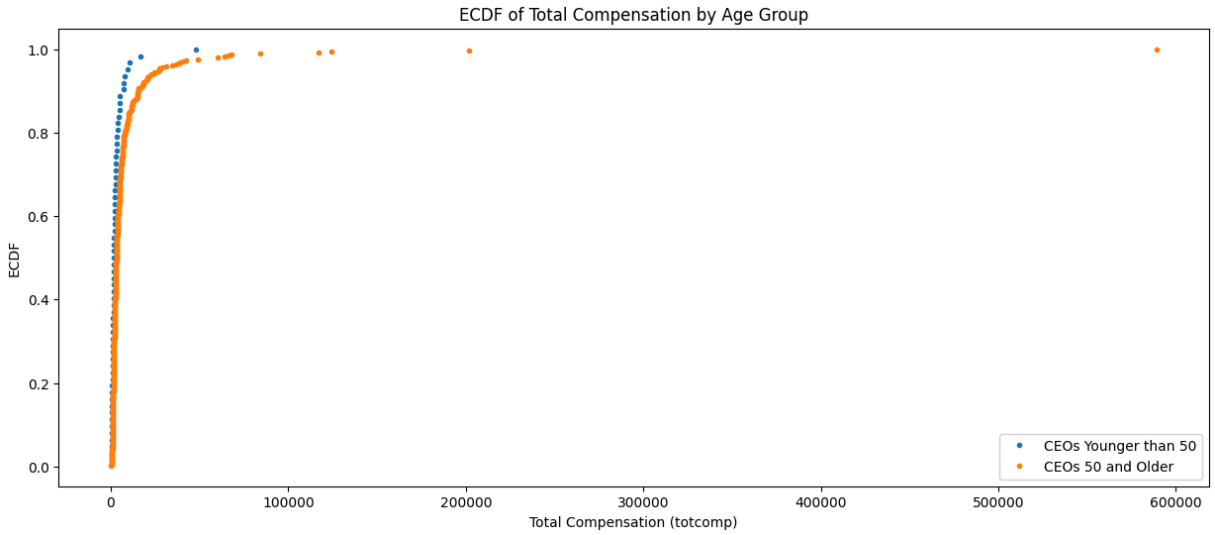
In [72]:
```
# Create two subsamples: CEOs younger than 50 and older than 50
ceo_younger_50 = data[data['age'] < 50]['totcomp']
ceo_older_50 = data[data['age'] >= 50]['totcomp']

# Find the 95th percentile for 'totcomp' to truncate the data
totcomp_95_quantile = data['totcomp'].quantile(0.95)

# Truncate the data to the 95th percentile for both subsamples
ceo_younger_50_truncated = ceo_younger_50[ceo_younger_50 <= totcomp_95_quant
ceo_older_50_truncated = ceo_older_50[ceo_older_50 <= totcomp_95_quantile]

# Plot overlapping histograms for both original and truncated data
plt.figure(figsize=(20, 6))

# Original data histograms
plt.subplot(1, 2, 1)
plt.hist(ceo_younger_50, bins='auto', alpha=0.6, label='CEOs Younger than 50
plt.hist(ceo_older_50, bins='auto', alpha=0.6, label='CEOs 50 and Older', ec
plt.title('Overlapping Histograms of Total Compensation by Age Group (Origir
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')
plt.legend()

# Truncated data histograms
plt.subplot(1, 2, 2)
plt.hist(ceo_younger_50_truncated, bins='auto', alpha=0.6, label='CEOs Young
plt.hist(ceo_older_50_truncated, bins='auto', alpha=0.6, label='CEOs 50 and
plt.title('Overlapping Histograms of Total Compensation by Age Group (Trunca
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('Frequency')
plt.legend()

plt.tight_layout()
plt.show()
```




The plots display the following:

- **Left Plot (Original Data):** This plot shows the total compensation distributions for both age groups without any truncation. The right-skewed nature of the data is evident, with a few very high compensation values.
- **Right Plot (Truncated to 95th Percentile):** This plot focuses on the compensation values up to the 95th percentile, removing the extreme outliers. This view reveals more details about the main body of the distribution for each age group.

In [73]:
```
# Plot Empirical Cumulative Distribution Functions (ECDFs) for both subsampl
plt.figure(figsize=(15, 6))

# ECDF for CEOs younger than 50
sorted_younger_50 = np.sort(ceo_younger_50)
ecdf_younger_50 = np.arange(1, len(sorted_younger_50) + 1) / len(sorted_your
plt.plot(sorted_younger_50, ecdf_younger_50, label='CEOs Younger than 50', m

# ECDF for CEOs older than 50
sorted_older_50 = np.sort(ceo_older_50)
ecdf_older_50 = np.arange(1, len(sorted_older_50) + 1) / len(sorted_older_50
plt.plot(sorted_older_50, ecdf_older_50, label='CEOs 50 and Older', marker='

plt.title('ECDF of Total Compensation by Age Group')
plt.xlabel('Total Compensation (totcomp)')
plt.ylabel('ECDF')
plt.legend()
plt.show()
```




In [81]:
```
# Compute location and dispersion measures for both subsamples
mean_younger_50 = np.mean(ceo_younger_50)
median_younger_50 = np.median(ceo_younger_50)
std_younger_50 = np.std(ceo_younger_50)
iqr_younger_50 = np.percentile(ceo_younger_50, 75) - np.percentile(ceo_young

mean_older_50 = np.mean(ceo_older_50)
median_older_50 = np.median(ceo_older_50)
std_older_50 = np.std(ceo_older_50)
iqr_older_50 = np.percentile(ceo_older_50, 75) - np.percentile(ceo_older_50,

age_group_table = PrettyTable()
age_group_table.field_names = ["Measure", "CEOs < 50", "CEOs ≥ 50"]
age_group_table.add_row(["Mean", f"{mean_younger_50:.2f}", f"{mean_older_50:
age_group_table.add_row(["Median", f"{median_younger_50:.2f}", f"{median_old
age_group_table.add_row(["Standard Deviation", f"{std_younger_50:.2f}", f"{s
age_group_table.add_row(["Interquartile Range (IQR)", f"{iqr_younger_50:.2f}

print(age_group_table)
```

```
+---------------------------+-----------+-----------+
|          Measure          | CEOs < 50 | CEOs ≥ 50 |
+---------------------------+-----------+-----------+
|            Mean           |  3459.85  |  9125.96  |
|           Median          |  1819.00  |  3161.00  |
|     Standard Deviation    |  6364.98  |  33819.12 |
| Interquartile Range (IQR) |  2167.25  |  4858.00  |
+---------------------------+-----------+-----------+
```

# Results and Interpretation

## Histograms

- The overlapping histograms show that the distribution of total compensation ( `totcomp` ) varies between CEOs younger than 50 and those 50 and older.
- The distribution for **CEOs 50 and older** appears more spread out, with a larger number of individuals receiving higher compensation, leading to a

right-skewed distribution.
- The **CEOs younger than 50** have a more concentrated distribution, with most compensations falling in a lower range.

### ECDFs

- The ECDF plots further illustrate the differences in distribution. The curve for **CEOs younger than 50** rises more steeply, indicating that a higher proportion of these CEOs have lower compensation.
- The ECDF for **CEOs 50 and older** increases more gradually, confirming that their compensation values are more spread out, with a significant proportion receiving higher compensation.

### Location and Dispersion Measures

- **Location Measures (Mean and Median):** The mean and median for CEOs 50 and older are both higher than those for younger CEOs, indicating that older CEOs generally receive higher total compensation.
- **Dispersion Measures (Std and IQR):** The standard deviation and IQR for older CEOs are significantly larger, indicating a greater spread in total compensation. This suggests that the compensation of older CEOs is more variable, with some receiving exceptionally high amounts.

## Conclusion

The differences in compensation distribution between the two age groups highlight a potential correlation between age and total compensation. Older CEOs not only tend to earn more on average, but their compensation also varies more widely.

The right-skewness and larger dispersion in the compensation of older CEOs may reflect greater diversity in career achievements, company sizes, and compensation structures for this age group.

# Task 3

> Consider another grouping of the data. Define the groups:

$$\begin{cases} S_1 & \text{if salary} < 3000 \\ S_2 & \text{if } 3000 \leq \text{salary} < 5000 \\ S_3 & \text{if salary} \geq 5000 \end{cases}$$

$$\begin{cases} A_1 & \text{if age} < 50 \\ A_2 & \text{if age} \geq 50 \end{cases}$$

In [75]:
```python
# Define groups based on salary and age as described
def define_salary_group(salary):
    if salary < 3000:
        return 'S1'
    elif 3000 <= salary < 5000:
        return 'S2'
    else:
        return 'S3'

def define_age_group(age):
    if age < 50:
        return 'A1'
    else:
        return 'A2'

# Apply these group definitions to the dataset
data['salary_group'] = data['salary'].apply(define_salary_group)
data['age_group'] = data['age'].apply(define_age_group)

data[['salary', 'age', 'salary_group', 'age_group']].head()
```

Out[75]:

| | salary | age | salary_group | age_group |
|---|---|---|---|---|
| **0** | 3030 | 61 | S2 | A2 |
| **1** | 6050 | 51 | S3 | A2 |
| **2** | 3571 | 63 | S2 | A2 |
| **3** | 3300 | 60 | S2 | A2 |
| **4** | 10000 | 63 | S3 | A2 |

# Part A

> Aggregate the data to a 2×3 contigency table with absolute and with relative frequencies.

In [80]:
```python
# Crosstab solution: contingency table with absolute frequencies
contingency_table_abs = pd.crosstab(data['age_group'], data['salary_group'])

# Crosstab solution: contingency table with relative frequencies
contingency_table_rel = pd.crosstab(data['age_group'], data['salary_group'],

# Piviot table solution: pandas pivot_table for absolute frequencies
contingency_table_abs_lib = pd.pivot_table(data, values='salary', index='age

# Piviot table solution: pandas pivot_table for relative frequencies
contingency_table_rel_lib = contingency_table_abs_lib / contingency_table_ab

abs_table = PrettyTable()
abs_table.field_names = ["Age Group", "S1", "S2", "S3"]
for index, row in contingency_table_abs.iterrows():
```

```
    abs_table.add_row([index] + list(row))

rel_table = PrettyTable()
rel_table.field_names = ["Age Group", "S1", "S2", "S3"]
for index, row in contingency_table_rel.iterrows():
    rel_table.add_row([index] + [f"{val:.4f}" for val in row])

print("Absolute Frequency Contingency Table:")
print(abs_table)
print("\nRelative Frequency Contingency Table:")
print(rel_table)
```

```
Absolute Frequency Contingency Table:
+-----------+-----+----+----+
| Age Group |  S1 | S2 | S3 |
+-----------+-----+----+----+
|     A1    |  59 | 3  | 0  |
|     A2    | 323 | 37 | 25 |
+-----------+-----+----+----+

Relative Frequency Contingency Table:
+-----------+--------+--------+--------+
| Age Group |   S1   |   S2   |   S3   |
+-----------+--------+--------+--------+
|     A1    | 0.1320 | 0.0067 | 0.0000 |
|     A2    | 0.7226 | 0.0828 | 0.0559 |
+-----------+--------+--------+--------+
```

# Part B

> Give interpretation for the values $n_{12}$ , $h_{12}$ , $n_1$ and $h_1$.

## Interpretations

- **( $n_{12}$ ):** Indicates the specific count of individuals who are both in a particular age group and a specific salary group. For instance, it could represent the number of CEOs younger than 50 who have a salary between 3000 and 5000.
- **( $h_{12}$ ):** Represents the proportion of individuals in the dataset that belong to this specific combination of age and salary group.
- **( $n_{1.}$ ):** Shows the total number of individuals in a specific age group, regardless of their salary.
- **( $h_{1.}$ ):** Provides the overall proportion of individuals in the specified age group within the entire dataset.

# Part C

> Compute an appropriate dependence measure for $S_i$ and $A_j$. What can be concludedfrom its value? What can we infer about the co-movement of the variables or theirchanges in opposite directions?

### Chi-Squared Formula

$$\chi^2 = \sum_{i=1}^{k} \sum_{j=1}^{l} \frac{(n_{ij} - \frac{n_{i.} \cdot n_{.j}}{n})^2}{\frac{n_{i.} \cdot n_{.j}}{n}}$$

### Contingency Coefficient of Pearson

$$C = \sqrt{\frac{\chi^2}{\chi^2 + n}}, \quad \text{with } C_{\max} = \sqrt{\frac{\min\{k, l\} - 1}{\min\{k, l\}}}$$

### Corrected Contingency Coefficient of Pearson

$$C_{\text{corr}} = \frac{C}{C_{\max}} \in [0, 1]$$

In [95]:
```
# Assuming `contingency_table_abs` is the DataFrame of the contingency table
# Calculate the chi-squared statistic, p-value, degrees of freedom, and expe
chi2, p, dof, ex = chi2_contingency(contingency_table_abs)
n = np.sum(contingency_table_abs.values)  # Total number of observations

# Calculate the Contingency Coefficient (C)
contingency_coefficient = np.sqrt(chi2 / (chi2 + n))

# Calculate the maximum value of the Contingency Coefficient (C_max)
min_dim = min(contingency_table_abs.shape)  # min(k, l)
contingency_coefficient_max = np.sqrt((min_dim - 1) / min_dim)

# Calculate the Corrected Contingency Coefficient (C_corr)
corrected_contingency_coefficient = contingency_coefficient / contingency_cc

print(f"Chi2: {chi2:.4f}")
print(f"Contingency Coefficient (C): {contingency_coefficient:.4f}")
print(f"Corrected Contingency Coefficient (C_corr): {corrected_contingency_c
```

```
Chi2: 6.1777
Contingency Coefficient (C): 0.1168
Corrected Contingency Coefficient (C_corr): 0.1651
```

## Conclusion from the Contingency Coefficients

1. **Chi-squared: 6.1777**
   - The chi-squared statistic is relatively low, suggesting that the observed frequencies in the contingency table do not deviate significantly from what would be expected under the assumption of independence. This

indicates a weak relationship between the age groups ($A_j$) and salary groups ($S_i$).

2. **Contingency Coefficient: 0.1168**

   - The contingency coefficient value of 0.1168 suggests a weak association between the age and salary groups. Since this coefficient is close to zero, it indicates that there is little to no dependency between these variables.
   - The weak association implies that changes in age groups do not significantly influence the distribution of salary groups and vice versa.

3. **Corrected Contingency Coefficient: 0.1651**

   - The corrected contingency coefficient of 0.1651, normalized to be between 0 and 1, reinforces the conclusion that there is only a minor dependency between age and salary groups. A value much closer to 1 would indicate a stronger relationship, while values near zero suggest independence.
   - This value confirms that the relationship between the variables is weak. There is no strong evidence of co-movement or significant influence in opposite directions between the age and salary categories.

The weak dependency observed in the coefficients implies that knowing a CEO's age group provides little information about their salary group. In economic terms, this suggests that factors other than age are likely more influential in determining a CEO's compensation.

# Problem 2: Descriptive Statistics and Probability Theory: Simulated Data

In [95]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import norm, spearmanr, pearsonr

from prettytable import PrettyTable

import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

## Task 1

In practice the data is always very heterogenous. To reflect it we contaminate the data by adding an outlier or a subsample with different characteristics.

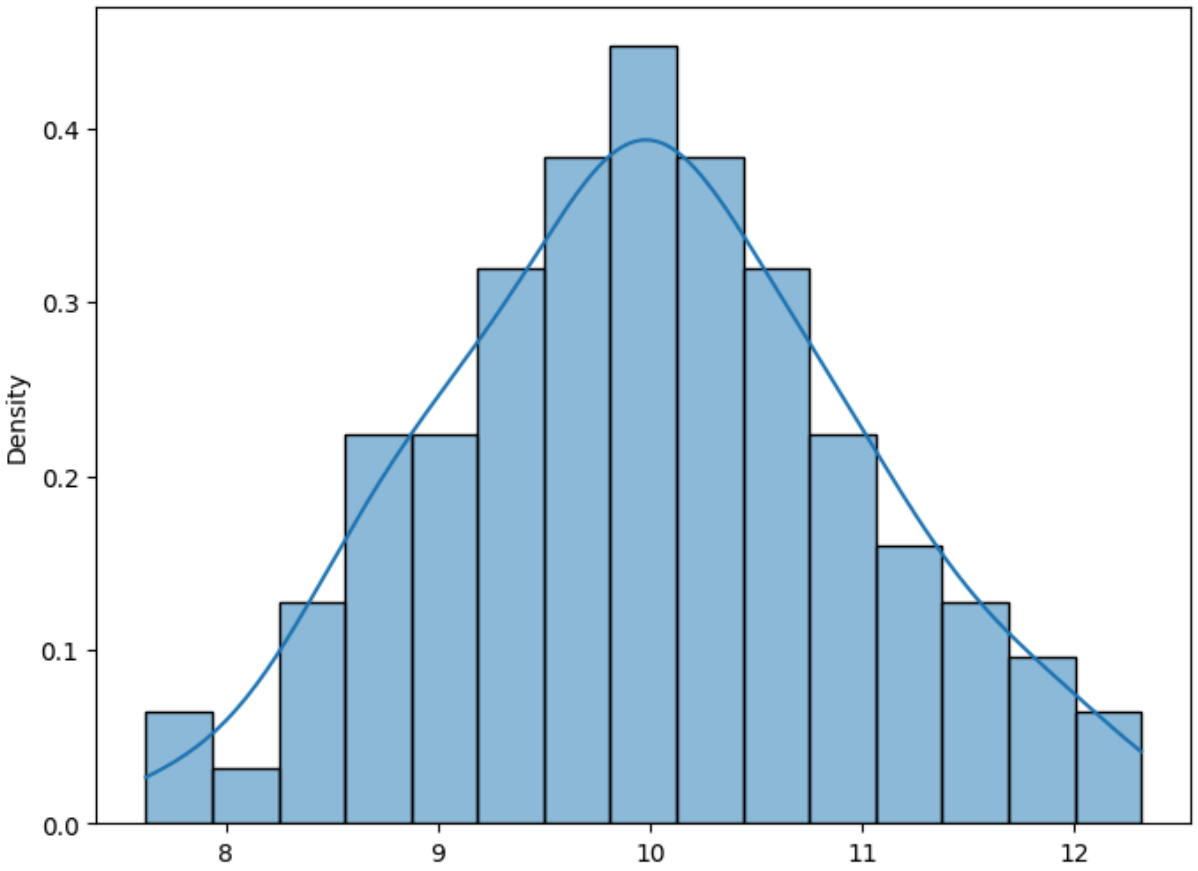
### Part A

> 1. Draw a sample of size 100 from N(10, $1^2$).
> 2. To obtain a realistic heterogeneous sample, add to this data a new sample of size *m* simulated from N(20, $2^2$), i.e. $\mu_2 = 20$ and ${\sigma_2}^2 = 4$.
>    - The size *m* will obviously influence the properties of the whole dataset.
>    - Vary *m* from 10 to 200 (by steps of 20).
>
>    (The resulting sample is said to stem from a mixture normal distribution and arises in practice if we put together observations from two homogenous cohorts.)
> 3. Calculate mean, median, and sample variance as a function of *m*.
> 4. Discuss the impact of heterogeneity on these measures.

In [14]:
```
# Step 1: Draw a sample of size 100 from N(10, 1^2)
sample_1 = np.random.normal(loc=10, scale=1, size=100)
```

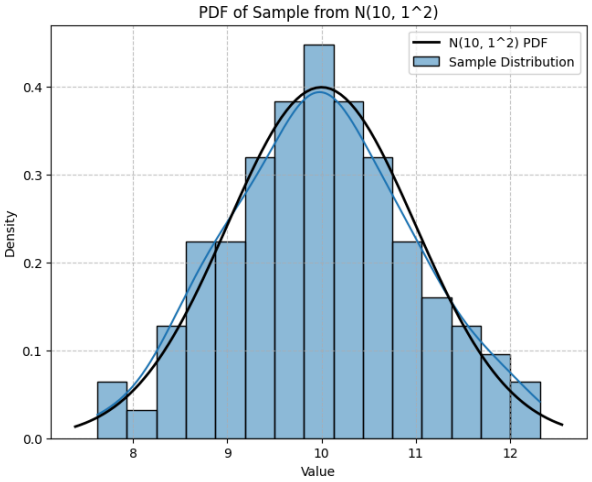
```
plt.figure(figsize=(8, 6))
sns.histplot(sample_1, bins=15, kde=True, stat='density', label='Sample Dist
```

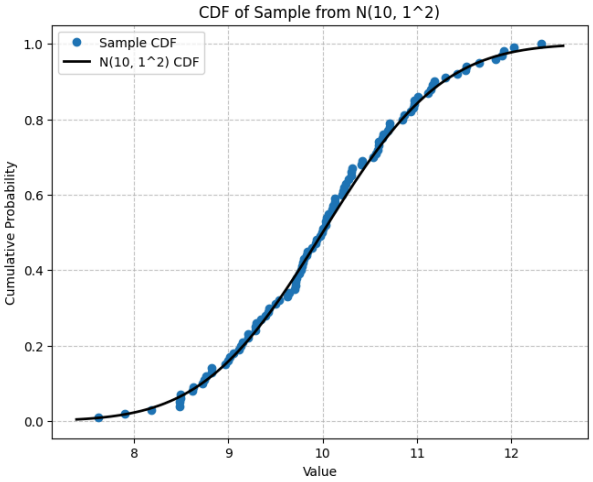
Out[14]: <Axes: ylabel='Density'>




In [33]:
```
# Plot the PDF and CDF
plt.figure(figsize=(14, 6))

# Plot PDF
plt.subplot(1, 2, 1)
sns.histplot(sample_1, bins=15, kde=True, stat='density', label='Sample Dist
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, 10, 1)  # N(10, 1^2) PDF
plt.plot(x, p, 'k', linewidth=2, label='N(10, 1^2) PDF')
plt.title('PDF of Sample from N(10, 1^2)')
plt.xlabel('Value')
plt.ylabel('Density')
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)

# Plot CDF
plt.subplot(1, 2, 2)
# Sort the sample for CDF
sample_sorted = np.sort(sample_1)
# Calculate CDF for the sorted sample
cdf_values = np.arange(1, len(sample_sorted) + 1) / len(sample_sorted)
plt.plot(sample_sorted, cdf_values, marker='o', linestyle='', label='Sample
```

```
# Overlay theoretical CDF of N(10, 1^2)
cdf_theoretical = norm.cdf(x, 10, 1)
plt.plot(x, cdf_theoretical, 'k-', linewidth=2, label='N(10, 1^2) CDF')

plt.title('CDF of Sample from N(10, 1^2)')
plt.xlabel('Value')
plt.ylabel('Cumulative Probability')
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)

# Adjust layout
plt.tight_layout(pad=3)
plt.show()
```




# Explanation of the Plots

## 1. Probability Density Function (PDF) Plot

The PDF represents how values are distributed within the sample drawn from $N(10, 1^2)$. The histogram shows the sample distribution, and the KDE (Kernel Density Estimate) provides a smooth approximation. We also plotted the theoretical PDF for $N(10, 1^2)$. The close match between the sample's KDE and the theoretical PDF confirms that the sample behaves as expected, with most values concentrated near the mean (10).

## 2. Cumulative Distribution Function (CDF) Plot

The CDF shows the cumulative probability up to a given value, indicating the probability that a random variable is less than or equal to a certain value. The sample's CDF, plotted using sorted values, closely follows the theoretical CDF for $N(10, 1^2)$, showing that the sample's cumulative behavior aligns with the expected normal distribution.

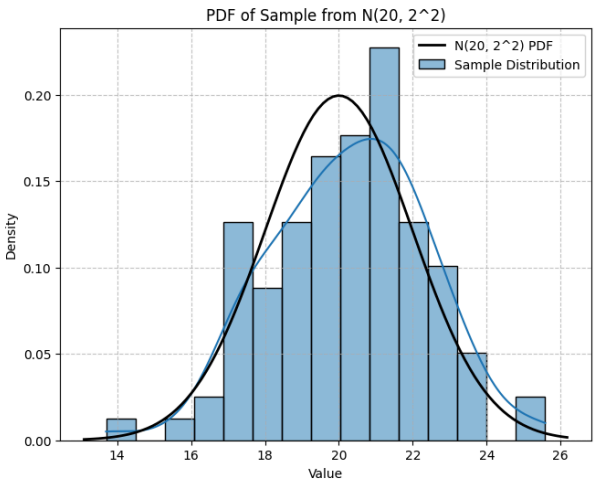
In [45]:
```
# Function to generate the heterogeneous sample for a given m
def generate_subsample(m):
    return np.random.normal(loc=20, scale=2, size=m)
```

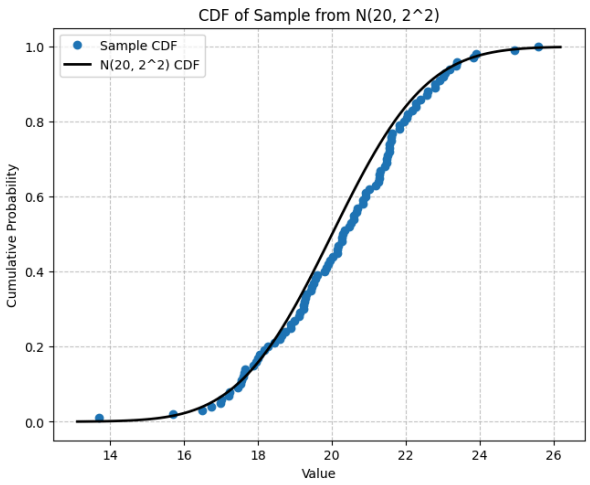
```
# Function to compute mean, median, and variance for a combined sample
def compute_statistics(m):
    # Draw a sample of size m from N(20, 2^2)
    sample_2 = generate_subsample(m)

    # Combine the two samples
    combined_sample = np.concatenate([sample_1, sample_2])

    # Calculate mean, median, and variance
    mean = np.mean(combined_sample)
    median = np.median(combined_sample)
    variance = np.var(combined_sample)

    return mean, median, variance
```

In [39]:

```
# Step 2: Draw a sample of size m from N(20, 2^2)
sample_2 = generate_subsample(100)

# Plot the PDF and CDF
plt.figure(figsize=(14, 6))

# Plot PDF
plt.subplot(1, 2, 1)
sns.histplot(sample_2, bins=15, kde=True, stat='density', label='Sample Dist
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, 20, 2)  # N(20, 2^2) PDF
plt.plot(x, p, 'k', linewidth=2, label='N(20, 2^2) PDF')
plt.title('PDF of Sample from N(20, 2^2)')
plt.xlabel('Value')
plt.ylabel('Density')
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)

# Plot CDF
plt.subplot(1, 2, 2)
# Sort the sample for CDF
sample_sorted = np.sort(sample_2)
# Calculate CDF for the sorted sample
cdf_values = np.arange(1, len(sample_sorted) + 1) / len(sample_sorted)
plt.plot(sample_sorted, cdf_values, marker='o', linestyle='', label='Sample

# Overlay theoretical CDF of N(20, 2^2)
cdf_theoretical = norm.cdf(x, 20, 2)
plt.plot(x, cdf_theoretical, 'k-', linewidth=2, label='N(20, 2^2) CDF')

plt.title('CDF of Sample from N(20, 2^2)')
plt.xlabel('Value')
plt.ylabel('Cumulative Probability')
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)

# Adjust layout
```

```
plt.tight_layout(pad=3)
plt.show()
```



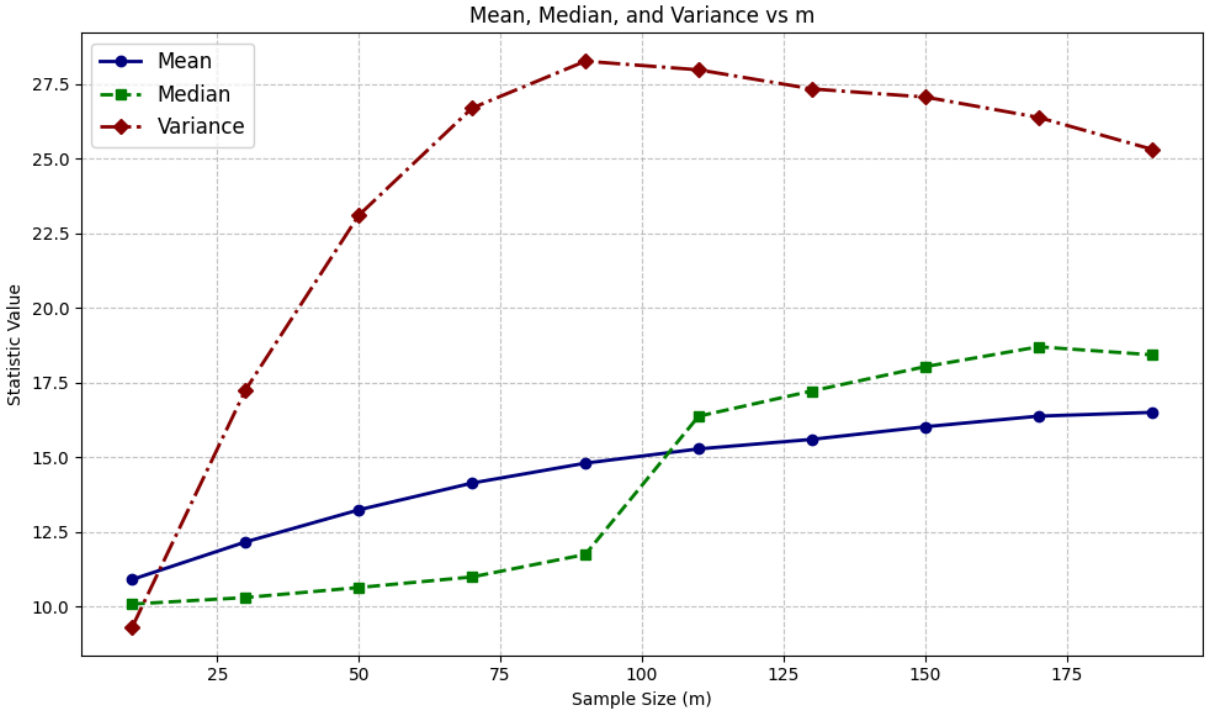




In [68]:
```
# Step 3: Vary m from 10 to 200 (steps of 20) and compute statistics for eac
m_values = range(10, 201, 20)
statistics = {'m': [], 'mean': [], 'median': [], 'variance': []}

for m in m_values:
    mean, median, variance = compute_statistics(m)
    statistics['m'].append(m)
    statistics['mean'].append(mean)
    statistics['median'].append(median)
    statistics['variance'].append(variance)

stats_df = pd.DataFrame(statistics)

table = PrettyTable()
table.field_names = ["m", "Mean", "Median", "Variance"]

for m, mean, median, variance in zip(statistics['m'], statistics['mean'], st
    table.add_row([m, round(mean, 2), round(median, 2), round(variance, 2)])

table_title = "Statistics for Different Sample Sizes (m)"
print(f"\n{table_title}\n")
print(table)
```

Statistics for Different Sample Sizes (m)

| m | Mean | Median | Variance |
|---|---|---|---|
| 10 | 10.91 | 10.08 | 9.32 |
| 30 | 12.17 | 10.3 | 17.26 |
| 50 | 13.24 | 10.64 | 23.11 |
| 70 | 14.14 | 10.99 | 26.7 |
| 90 | 14.8 | 11.75 | 28.27 |
| 110 | 15.28 | 16.38 | 27.98 |
| 130 | 15.6 | 17.22 | 27.34 |
| 150 | 16.02 | 18.04 | 27.07 |
| 170 | 16.38 | 18.7 | 26.39 |
| 190 | 16.5 | 18.43 | 25.32 |

In [69]:
```
# Step 4: Plot the results to visualize the impact of m on the statistic
plt.figure(figsize=(10, 6))

# Plot mean
plt.plot(stats_df['m'], stats_df['mean'], marker='o', color='navy', linestyl

# Plot median
plt.plot(stats_df['m'], stats_df['median'], marker='s', color='green', lines

# Plot variance
plt.plot(stats_df['m'], stats_df['variance'], marker='D', color='darkred', l

plt.title('Mean, Median, and Variance vs m')
plt.xlabel('Sample Size (m)')
plt.ylabel('Statistic Value')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(fontsize=12)
plt.tight_layout()
plt.show()
```




## Impact of Heterogeneity on the Mean, Median, and Variance

### Key Observations:

- **Mean:**
  Initially, the mean of the combined sample is close to 10 (the mean of the first sample drawn from $N(10, 1^2)$). As $m$ increases, the influence of the second distribution $N(20, 2^2)$ grows, pulling the mean upward. The mean gradually shifts toward 20 as more samples from the second distribution are added, showing how the overall data's central tendency changes with increased heterogeneity.

- **Median:**
  The median follows a similar pattern to the mean but with more sensitivity to the changes in the distribution. Around $m = 100$, the median shows a sudden increase, indicating that the second distribution's samples are starting to dominate the central tendency of the combined sample. The irregular shape of the median compared to the mean suggests that the median is more influenced by the uneven distribution of values as more samples from $N(20, 2^2)$ are added.

- **Variance:**
  The variance increases sharply as $m$ grows. This is because the second distribution $N(20, 2^2)$ has a larger variance (4 compared to 1 in $N(10, 1^2)$). As more samples are added from the higher-variance distribution, the overall

variability of the combined sample increases. After $m \approx 90$, the variance stabilizes, indicating that the second distribution is beginning to dominate the combined sample, and further increases in $m$ no longer cause significant changes in variance.

### Summary of the Impact of Heterogeneity:

**Heterogeneity** (mixing two distributions) significantly alters the mean, median, and variance of the combined sample. The **mean** and **median** shift upward, moving towards the central value of the second distribution. The mean changes more gradually, while the median reacts more abruptly around $m = 100$, reflecting the influence of the second distribution's central tendency. The **variance** increases rapidly, as the second distribution introduces more variability into the combined sample, causing the overall variance to grow until it stabilizes.

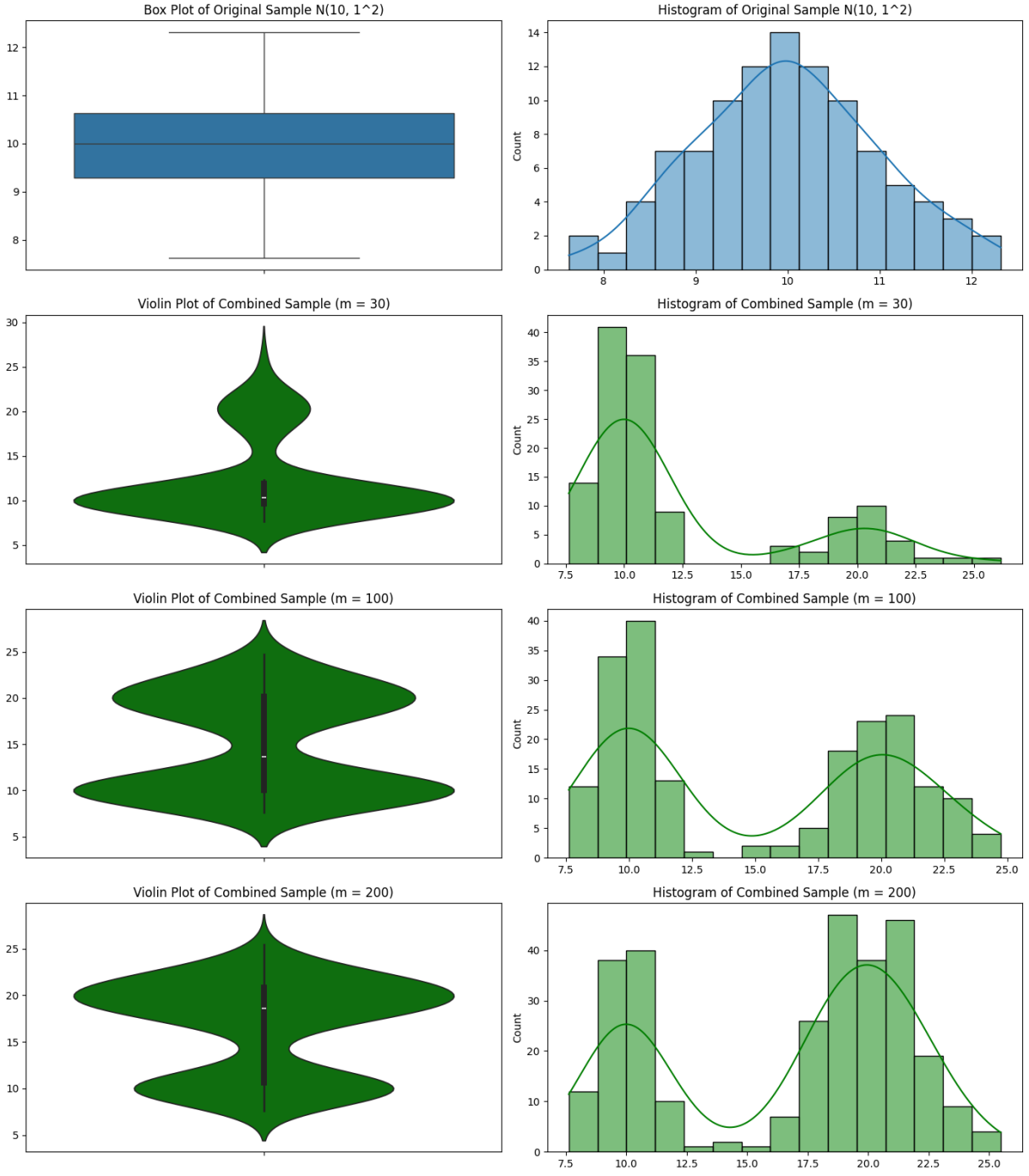
As $m$ increases, the characteristics of the second distribution begin to dominate, altering the core statistics of the overall dataset.

## Part B

> Plot Box-plots (or violin plots) and histograms for each subsample individually and for the sample for a few different values of $m$.

In [92]:
```
# Values of m to explore
m_values = [30, 100, 200]

# Step 1: Generate subsamples for different m values
subsamples = {m: generate_subsample(m) for m in m_values}

# Step 2: Plot histograms and violin/box plots
plt.figure(figsize=(14, 16))

plt.subplot(4, 2, 1)
sns.boxplot(data=sample_1)
plt.title('Box Plot of Original Sample N(10, 1^2)')

plt.subplot(4, 2, 2)
sns.histplot(sample_1, bins=15, kde=True)
plt.title('Histogram of Original Sample N(10, 1^2)')

subplot_index = 3
for m in m_values:
    combined_sample = np.concatenate([sample_1, subsamples[m]])

    # Plot the Box plot for each subsample + combined sample
    plt.subplot(4, 2, subplot_index)
    sns.violinplot(data=combined_sample, color='green')
    plt.title(f'Violin Plot of Combined Sample (m = {m})')
```

```
    # Plot the Histogram for each combined sample
    plt.subplot(4, 2, subplot_index + 1)
    sns.histplot(combined_sample, bins=15, kde=True, color='green')
    plt.title(f'Histogram of Combined Sample (m = {m})')

    subplot_index += 2

plt.tight_layout()
plt.show()
```




## Statistical Analysis of Sample and Subsamples

## 1. Original Sample ($N(10, 1^2)$)

- **Box Plot:**
  The box plot of the original sample shows a symmetrical distribution centered around the mean of 10, which is typical for a normal distribution with mean 10 and variance 1. The interquartile range (IQR) is compact, and there are no visible outliers, indicating a well-contained spread.

- **Histogram:**
  The histogram confirms that the sample is normally distributed, showing a bell-shaped curve. The majority of values cluster around 10, with a few falling within the range of approximately 8 to 12, which aligns with the sample's standard deviation of 1.

## 2. Combined Samples with Different Values of m

As we introduce subsamples with different values of $m$, the combined distributions reflect increasing heterogeneity, as seen in both the violin plots and histograms.

### m = 30

- **Violin Plot:**
  The violin plot shows a clear bimodal distribution. The original sample's values (around 10) are still evident, but the addition of the second distribution (from the subsample) introduces a secondary peak, which reflects the increased variability in the combined dataset.

- **Histogram:**
  The histogram shows that the data is now more spread out. While there is still a peak at around 10, indicating the influence of the original sample, the second peak (around 20) begins to emerge as $m = 30$ introduces a more diverse set of values. The overall spread is wider, ranging from approximately 7.5 to 22.5.

### m = 100

- **Violin Plot:**
  The bimodal pattern becomes more pronounced as the subsample grows larger. The original and subsample distributions each contribute to distinct peaks, which further widens the spread of the combined dataset.

- **Histogram:**
  The histogram shows an even wider spread, with a more substantial peak forming at around 20. As $m$ increases, the second distribution's influence

becomes stronger, shifting the center of the combined sample. The values are now more evenly spread between the ranges of 7.5 and 25.

### m = 200

- **Violin Plot:**
  At *m = 200*, the violin plot shows a very symmetric bimodal distribution. The two peaks (around 10 and 20) become almost equal, indicating that the original sample's influence is now balanced by the subsample's distribution. This symmetry demonstrates that the combined dataset is now a mix of two distributions.

- **Histogram:**
  The histogram further supports this observation by showing that the combined distribution is now heavily influenced by the second distribution. The data is spread widely, with peaks forming at around 10 and 20, and the overall distribution ranging from approximately 7.5 to 25. The combined sample is now highly heterogeneous, reflecting both sources.

### Conclusion

As *m* increases, the heterogeneity of the combined dataset becomes more evident. The violin plots and histograms show how the second distribution gradually takes over, spreading the data more widely and shifting the overall central tendency. The variance increases, and the influence of the original sample diminishes as the subsample size grows. These visualizations demonstrate the impact of combining distributions with different means and variances, resulting in a more complex overall distribution.

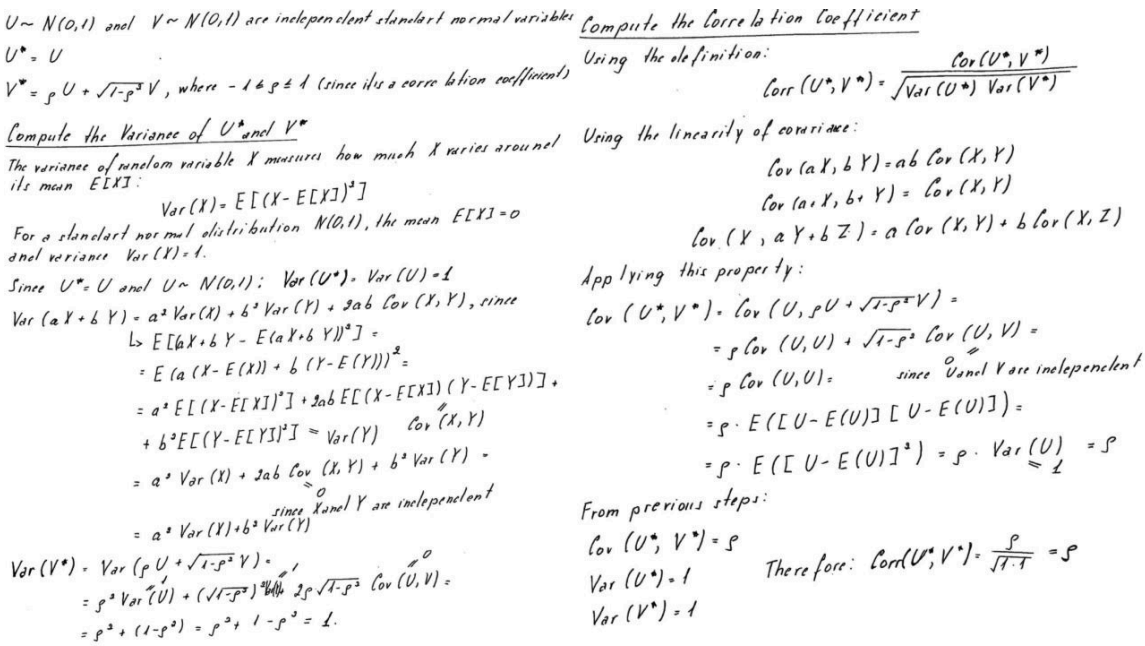
# Task 2

Next step is to simulate two dependent data sets. We simulate two samples with a given value of the correlation coefficient.

# Part A

> Let $U \sim N(0, 1)$ and $V \sim N(0, 1)$. Let $U^* = U$ and $V^* = \rho U + \sqrt{(1 - \rho^2)}V$. Prove that $Corr(U^*, V^*) = \rho$ and the variances of both variables $U^*$ and $V^*$ equal one.

## Manual Solution




# Implementation in Code

## Key Steps:

- Simulate $U$ and $V$ as independent standard normal random variables.
- Construct $U^* = U$ and $V^* = \rho U + \sqrt{(1 - \rho^2)}V$.
- Compute the variance of $U^*$ and $V^*$.
- Compute the correlation between $U^*$ and $V^*$.
- Plot the distributions of $U^*$ and $V^*$.

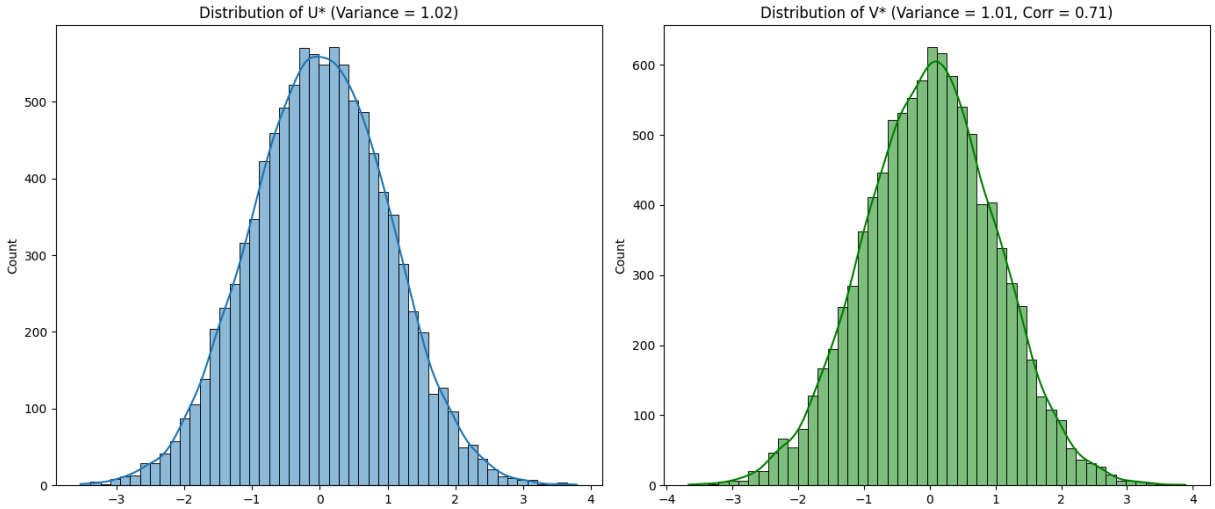
## Expected Output:

- Variance of $U^*$ should be 1 (since $U^* = U$).
- Variance of $V^*$ should also be 1 based on the transformation.
- Correlation between $U^*$ and $V^*$ should be approximately $\rho = 0.7$.

In [94]:
```
# Parameters
rho = 0.7  # Correlation coefficient
n = 10000  # Number of samples

# Step 1: Simulate independent standard normal variables U and V
U = np.random.normal(0, 1, n)
V = np.random.normal(0, 1, n)

# Step 2: Construct U* and V* based on the given formulas
U_star = U
V_star = rho * U + np.sqrt(1 - rho**2) * V

# Step 3: Compute the variances and correlation
var_U_star = np.var(U_star)
```

```
var_V_star = np.var(V_star)
corr_U_star_V_star = np.corrcoef(U_star, V_star)[0, 1]

# Print the computed values
print(f"Variance of U*: {var_U_star:.2f}")
print(f"Variance of V*: {var_V_star:.2f}")
print(f"Correlation between U* and V*: {corr_U_star_V_star:.2f}")

# Step 4: Plot the distributions of U* and V*
plt.figure(figsize=(14, 6))

# Distribution of U*
plt.subplot(1, 2, 1)
sns.histplot(U_star, bins=50, kde=True)
plt.title(f'Distribution of U* (Variance = {var_U_star:.2f})')

# Distribution of V*
plt.subplot(1, 2, 2)
sns.histplot(V_star, bins=50, kde=True, color='green')
plt.title(f'Distribution of V* (Variance = {var_V_star:.2f}, Corr = {corr_U_

plt.tight_layout()
plt.show()
```

```
Variance of U*: 1.02
Variance of V*: 1.01
Correlation between U* and V*: 0.71
```




# Part B

> Use the above idea to simulate two samples of size $n = 100$ from a normal distribution with different values of $\rho$. Compute the correlation coefficients of Pearson and of Spearman. Compare the correlation to the original parameter $\rho$ (for example, plot Pearson vs. $\rho$ and Spearman vs. $\rho$).

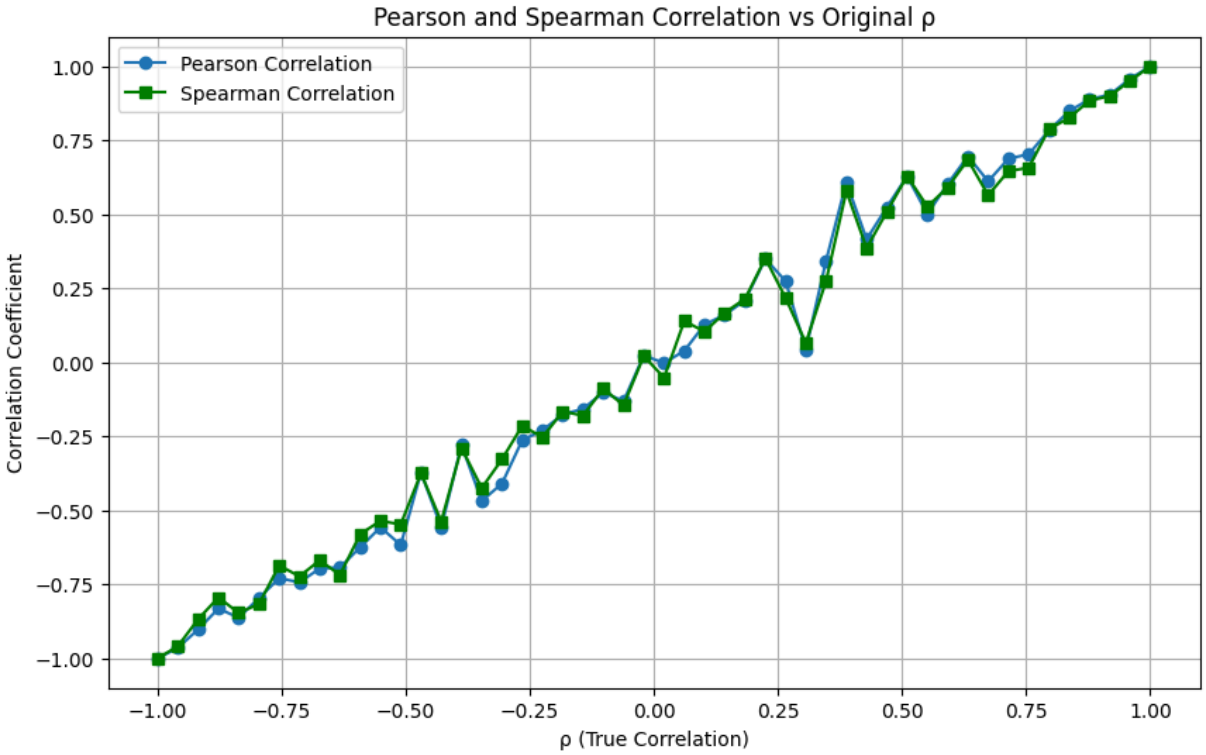
Key steps:

- We'll simulate different values of $\rho$ (correlation) in the range $[-1, 1]$.
- For each $\rho$, we will compute the corresponding Pearson and Spearman correlation coefficients for the samples $U^*$ and $V^*$.
- Finally, we'll plot the correlation coefficients (Pearson vs $\rho$ and Spearman vs $\rho$).

### Expected Output:

- The plot will show how closely the **Pearson correlation** follows the true $\rho$ values, and how the **Spearman correlation** behaves relative to both $\rho$ and Pearson's correlation.

In [97]:
```
# Parameters
n = 100  # Sample size
rho_values = np.linspace(-1, 1, 50)  # Different values of rho from -1 to 1
pearson_corrs = []
spearman_corrs = []

# Step 1: Simulate the samples for different values of rho
for rho in rho_values:
    # Generate independent normal variables U and V
    U = np.random.normal(0, 1, n)
    V = np.random.normal(0, 1, n)

    # Construct U* and V* according to the formulas
    U_star = U
    V_star = rho * U + np.sqrt(1 - rho**2) * V

    # Step 2: Compute Pearson and Spearman correlations
    pearson_corr, _ = pearsonr(U_star, V_star)
    spearman_corr, _ = spearmanr(U_star, V_star)

    pearson_corrs.append(pearson_corr)
    spearman_corrs.append(spearman_corr)

# Step 3: Plot Pearson vs rho and Spearman vs rho
plt.figure(figsize=(10, 6))

# Pearson correlation plot
plt.plot(rho_values, pearson_corrs, label='Pearson Correlation', marker='o')
# Spearman correlation plot
plt.plot(rho_values, spearman_corrs, label='Spearman Correlation', color='gr

plt.title('Pearson and Spearman Correlation vs Original ρ')
plt.xlabel('ρ (True Correlation)')
plt.ylabel('Correlation Coefficient')
plt.legend()

plt.grid(True)
plt.show()
```




# Part C

> Make a nonlinear but monotone transformation of $V^*$, say *exp* for simplicity. Check the impact of this transformation on the correlation coefficients of Spearman and Pearson. Think about an appropriate visualization of the findings.

## Key Steps:

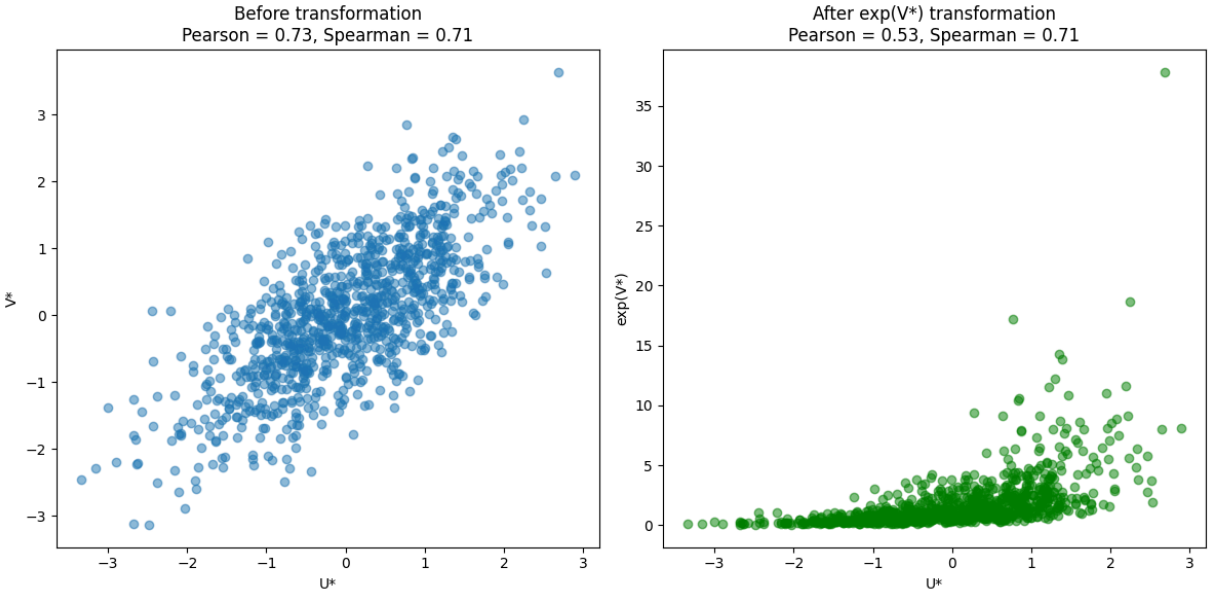
- Simulate two variables $U^*$ and $V^*$.
- Apply the exponential transformation to $V^*$.
- Compute and compare the Pearson and Spearman correlation coefficients before and after the transformation.
- Visualize the differences in correlation coefficients.
- Plot histograms of $V^*$ and $exp(V^*)$ to visually inspect the distribution before and after the transformation.
- Apply different transformations to $V^*$ (like $V^*$, $exp(V^*)$, $V^{*2}$, etc.) and plot the corresponding Pearson and Spearman correlations to observe trends.

In [107…

```
# Parameters
rho = 0.7  # Correlation coefficient
n = 1000  # Sample size

# Step 1: Simulate independent normal variables U and V
U = np.random.normal(0, 1, n)
ndom.normal(0, 1, n)
```

```
# Construct U* and V* based on the given formulas
U_star = U
V_star = rho * U + np.sqrt(1 - rho**2) * V

# Step 2: Apply a nonlinear transformation (exponential) to V*
V_star_exp = np.exp(V_star)

# Step 3: Compute Pearson and Spearman correlations
pearson_before, _ = pearsonr(U_star, V_star)
spearman_before, _ = spearmanr(U_star, V_star)

pearson_after, _ = pearsonr(U_star, V_star_exp)
spearman_after, _ = spearmanr(U_star, V_star_exp)

# Print the results
print(f"Before transformation: Pearson = {pearson_before:.2f}, Spearman = {s
print(f"After transformation: Pearson = {pearson_after:.2f}, Spearman = {spe

# Step 4: Visualize the distributions and impact on correlation

# Plot before and after the transformation
plt.figure(figsize=(12, 6))

# Plot U* vs V* before transformation
plt.subplot(1, 2, 1)
plt.scatter(U_star, V_star, alpha=0.5)
plt.title(f'Before transformation\nPearson = {pearson_before:.2f}, Spearman
plt.xlabel('U*')
plt.ylabel('V*')

# Plot U* vs V* after transformation
plt.subplot(1, 2, 2)
plt.scatter(U_star, V_star_exp, alpha=0.5, color='green')
plt.title(f'After exp(V*) transformation\nPearson = {pearson_after:.2f}, Spe
plt.xlabel('U*')
plt.ylabel('exp(V*)')

plt.tight_layout()
plt.show()
```

```
Before transformation: Pearson = 0.73, Spearman = 0.71
After transformation: Pearson = 0.53, Spearman = 0.71
```




The plot compares the relationship between $U^*$ and $V^*$ before and after applying the exponential transformation to $V^*$. The Pearson and Spearman correlation coefficients are displayed for both cases.

## 1. Before Transformation:

- **Pearson Correlation:** The Pearson correlation is 0.73, indicating a strong linear relationship between $U^*$ and $V^*$. This is expected since the variables are generated with a linear relationship based on a correlation coefficient of $\rho = 0.7$.
- **Spearman Correlation:** The Spearman correlation is also 0.71, showing that the rank-based correlation is consistent with the linear relationship observed in the scatter plot. Spearman correlation is based on ranks, and this high value indicates that the rank order is preserved.
- **Interpretation:** The scatter plot shows a clear linear trend, with data points aligned along a diagonal, supporting the high Pearson correlation. The Spearman correlation being close to the Pearson value suggests the ranks of the values are also well-preserved.

## 2. After Exponential Transformation:

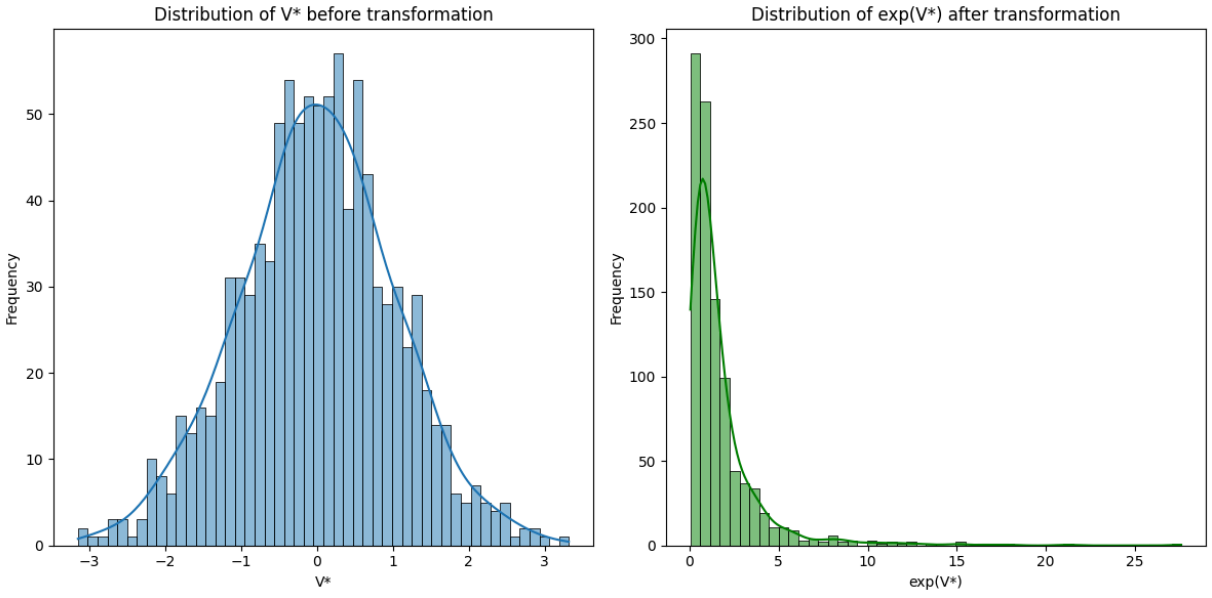
- **Pearson Correlation:** The Pearson correlation decreases to 0.53. This shows that the linearity between $U^*$ and $exp(V^*)$ has weakened due to the nonlinear nature of the exponential transformation.
- **Spearman Correlation:** The Spearman correlation remains at 0.71, demonstrating that the rank order is still preserved, even though the linear relationship is altered. This is expected because Spearman correlation only considers rank order, which remains unchanged after a monotonic transformation like exponentiation.

- **Interpretation:** The scatter plot shows a much different pattern after the exponential transformation. The data points are compressed near the lower values on the y-axis, reflecting the exponential scaling of values. Despite this distortion in the linear relationship, the Spearman correlation remains stable as the relative ranking of the data points has not changed.

In conclusion, the exponential transformation reduces the Pearson correlation by distorting the linear relationship, while the Spearman correlation remains stable as it depends on rank order, which is preserved by the monotonic transformation.

In [103…

```
plt.figure(figsize=(12, 6))

# Histogram of V* before transformation
plt.subplot(1, 2, 1)
sns.histplot(V_star, bins=50, kde=True)
plt.title('Distribution of V* before transformation')
plt.xlabel('V*')
plt.ylabel('Frequency')

# Histogram of V* after exp transformation
plt.subplot(1, 2, 2)
sns.histplot(V_star_exp, bins=50, kde=True, color='green')
plt.title('Distribution of exp(V*) after transformation')
plt.xlabel('exp(V*)')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```




The plot shows the distribution of $V^*$ before and after applying the exponential transformation.

### 1. Distribution of $V^*$ Before Transformation:

- **Shape:** The original distribution of $V^*$ is approximately normal, centered around 0, and symmetric. This is consistent with the fact that $V^*$ was generated from a linear combination of independent normal variables.
- **Spread:** The values are spread between approximately -3 and 3, indicating a relatively narrow range around the mean.
- **Interpretation:** This normal distribution reflects the inherent linear relationship between $U^*$ and $V^*$ before applying any transformations.

### 2. Distribution of $exp(V^*)$ After Transformation:

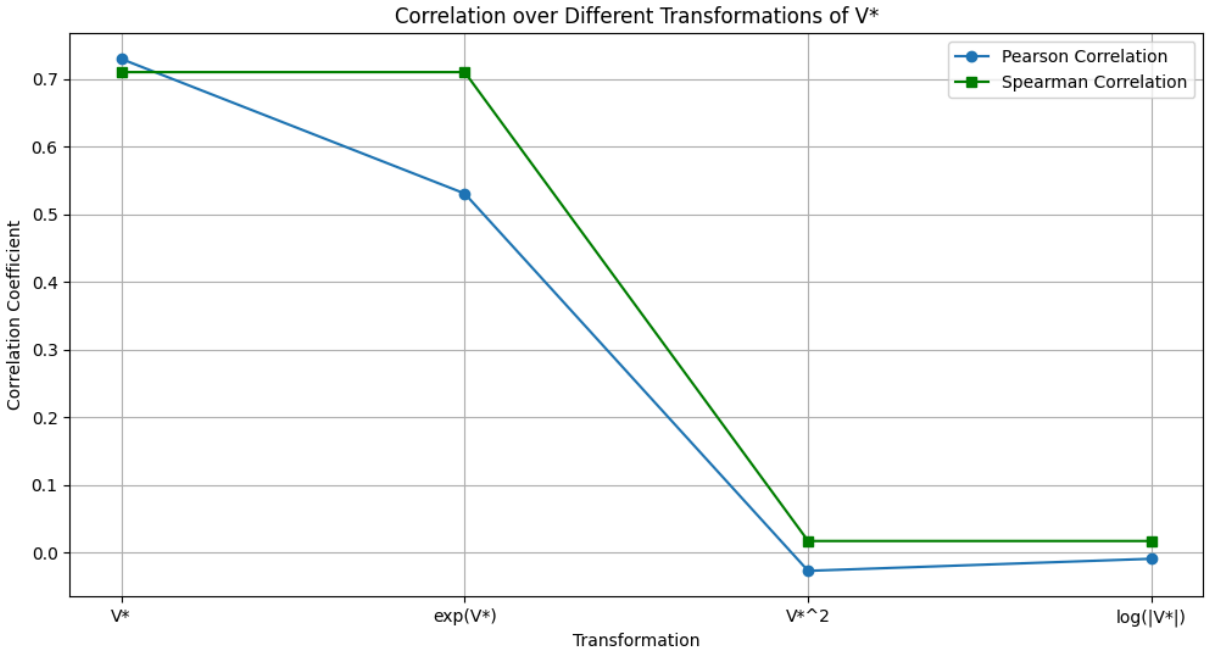
- **Shape:** After applying the exponential transformation, the distribution becomes right-skewed. This is a typical effect of applying an exponential function to a normal distribution, where negative values of $V^*$ map to values between 0 and 1, and positive values of $V^*$ get amplified.
- **Spread:** The values of $exp(V^*)$ are now concentrated between 0 and 15, with a long tail extending to larger values. Most of the mass is concentrated around smaller values, which represents the exponential mapping of smaller or negative values of $V^*$.
- **Interpretation:** The transformation drastically changes the distribution of the variable. While the rank order of the values remains unchanged (preserving Spearman correlation), the linear relationship is altered (affecting Pearson correlation). This is evident from the shift from a symmetric to a skewed distribution.

In summary, the exponential transformation changes the distribution of $V^*$ from a symmetric normal distribution to a right-skewed distribution, which affects linearity but preserves the rank order.

In [108…

```
# Apply different transformations to V*
transformations = {
    'V*': V_star,
    'exp(V*)': np.exp(V_star),
    'V*^2': V_star**2,
    'log(|V*|)': np.log(np.abs(V_star) + 1)  # Adding 1 to avoid log(0)
}

pearson_corrs = []
spearman_corrs = []

# Calculate Pearson and Spearman correlations for each transformation
for label, transformed_V_star in transformations.items():
    pearson_corr, _ = pearsonr(U_star, transformed_V_star)
    spearman_corr, _ = spearmanr(U_star, transformed_V_star)
    pearson_corrs.append(pearson_corr)
    man_corrs.append(spearman_corr)
```

```
# Plot Pearson and Spearman correlations over transformations
plt.figure(figsize=(12, 6))
plt.plot(transformations.keys(), pearson_corrs, label='Pearson Correlation',
plt.plot(transformations.keys(), spearman_corrs, label='Spearman Correlation
plt.title('Correlation over Different Transformations of V*')
plt.ylabel('Correlation Coefficient')
plt.xlabel('Transformation')
plt.legend()
plt.grid(True)
plt.show()
```




From the plot, we can observe how the Pearson and Spearman correlation coefficients are affected by different transformations of $V^*$. Here are the key takeaways:

## 1. Original $V^*$:

- **Pearson Correlation**: The Pearson correlation is quite high (around 0.7), as expected since the linear relationship between $U^*$ and $V^*$ matches the original correlation coefficient $\rho$.
- **Spearman Correlation**: The Spearman correlation is also close to 0.7, indicating that the rank correlation is consistent with the linear relationship.

## 2. Exponential Transformation $exp(V^*)$:

- **Pearson Correlation**: After applying the exponential transformation, the Pearson correlation decreases slightly, though it remains relatively high. This is because the transformation is monotonic but introduces some non-linearity, affecting the linear relationship.

- **Spearman Correlation**: The Spearman correlation remains almost the same, since Spearman correlation is based on ranks, and the exponential transformation preserves the rank order.

### 3. Quadratic Transformation $V^{*2}$:

- **Pearson Correlation**: The Pearson correlation drops sharply, approaching 0. This is because the quadratic transformation distorts the linear relationship significantly. As a result, the linear association between $U^*$ and $V^{*2}$ is almost lost.
- **Spearman Correlation**: The Spearman correlation also drops significantly, although not as drastically as the Pearson correlation. The quadratic transformation does distort the rank order of the variables, but the effect is less severe on ranks than on the linear relationship.

### 4. Logarithmic Transformation $log(|V^*|)$:

- **Pearson Correlation**: After applying the logarithmic transformation, the Pearson correlation remains very low, indicating that the linear relationship has been heavily distorted by the transformation.
- **Spearman Correlation**: The Spearman correlation remains close to 0. This is likely because the log transformation of the absolute value of $V^*$ alters the rank order in a non-trivial way, significantly affecting the monotonic relationship.

### Conclusions:

- **Pearson correlation** is highly sensitive to nonlinear transformations that distort the linear relationship, as seen with $V^{*2}$ and $log(|V^*|)$. Even monotonic transformations like $exp(V^*)$ can slightly affect the Pearson correlation due to the introduction of non-linearity.
- **Spearman correlation** is more robust to monotonic transformations, such as $exp(V^*)$, but still reacts to transformations like $V^{*2}$ and $log(|V^*|)$, which affect the rank order of the data.

This plot demonstrates the sensitivity of Pearson correlation to transformations that introduce non-linearity, while the Spearman correlation remains stable under monotonic transformations but is affected when the rank order is altered.

# Problem 3: Descriptive Statistics and Probability Theory: Probabilities and Distributions

In [21]:
```
import yfinance as yf
import numpy as np
import pandas as pd
from scipy.stats import norm
import scipy.stats as stats
import matplotlib.pyplot as plt
```

## Task 1

- ( A ): a randomly chosen student belongs to group A
- ( B ): a randomly chosen student belongs to group B
- ( E ): a randomly chosen student passes the exam

> (a) What is the probability that a randomly chosen student passes the exam AND belongs to group A?

> All students belong to one of two groups: well prepared for the exam (group A) and not so well prepared for the exam (group B). A "well prepared" student passes the exam with probability of 80% and a "not so well prepared"-student with probability of 30%. We assume that only 10% of all students belong to group B.

> (b) What is the probability that a randomly chosen student passes the exam?

> (c) What is the probability that a student who passed the exam, belongs to group B?

1. A: a randomly chosen student belongs to group A
B: a randomly chosen student belongs to group B
E: a randomly chosen student passes the exam

$P(A) = 1 - P(B) = 0,90$ (since 10% are in group B)

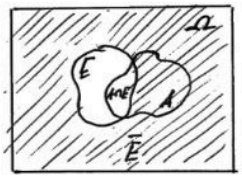
$P(B) = 0,10$

$P(E/A) = 0,80$ (well-prepared student passes)

$P(E/B) = 0,30$ (not-so-well-prepared student passes)

a) Probability that a randomly student passes the exam and belongs to group A:

$P(E \cap A)$ we can't say, that these variables are (stochastically) independent, thus $P(E \cap A) \neq P(A) \cdot P(B)$ in this case



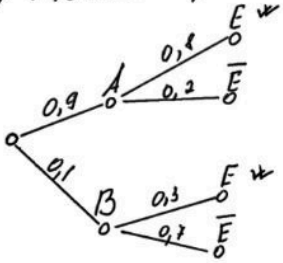

Relying on the conditional probability formula: $P(A/B) = \frac{P(A \cap B)}{P(B)}$ :

$$P(E \cap A) = P(E/A) \cdot P(A)$$

$P(E \cap A) = 0,80 \cdot 0,90 \approx 0,72$

Answer: 0,72 (or 72%)

b) Probability that a randomly chosen student passes the exam:




Relying on the law of total probability:

$$P(B) = \sum_{i=1}^{k} P(B/A_i) \cdot P(A_i)$$

$P(E) = P(E/A) \cdot P(A) + P(E/B) \cdot P(B) =$

$= 0,8 \cdot 0,9 + 0,3 \cdot 0,1 = 0,72 + 0,03 \approx 0,75$

Answer: 0,75 (or 75%)

c) Probability that a student who passed the exam belongs to group B?

$P(B) = 0,1$ – prior probability (any randomly chosen student belongs to group B before considering whether they passed the exam)

$P(B/E) = ?$ – posterior probability (a student belongs to group B after knowing they passed the exam)

Relying on the Bayes' Theorem:

(basic form) $P(A/B) = \frac{P(B/A) \cdot P(A)}{P(B)}$

A → evidence, B → hypothesis

$P(B/E) = \frac{P(E/B) \cdot P(B)}{P(E)}$

likelihood · prior probability / marginal probability

posterior probability

$P(\text{student belongs to group } B \text{ knowing they passed the exam}) = \frac{P(\text{student from group } B \text{ passes the exam}) \cdot P(\text{student is from group } B)}{P(\text{randomly chosen student passes the exam, regardless of which group they belong to})}$

$P(B/E) = \frac{0,3 \cdot 0,1}{0,75} = \frac{0,03}{0,75} \approx 0,04$

0,75 – calculated in part b

Answer: 0,04 (or 4 %)

# Task 2

> The dean and the head of the program invite all students to a barbecue party at the end of the year. Five vegetarians are among the guests. The pitmaster is an expert in juicy steaks and still needs some practice with grilled vegetables. For this reason, a portion of vegetables is burned to ashes with a probability of 60%.

> (a) What is the probability that none of the five vegetable servings is charred?

> (b) What is the probability that exactly two of the five vegetable servings are charred?

> (c) What is the probability that at least three out of five servings are charred?

2. The dean and the head of the program invite all students to a barbecue party at the end of the year. Five vegetarians are among the guests. The pitmaster is an expert in juicy steaks and still need some practice with grilled vegetables. For this reason a portion of vegetables is burned to ashes with probability of 60%.

- There are 5 vegetarians
- The probability that a portion of vegetables is burned is 60%
  $p = 0.60$
- The probability that a portion is not burned is $q = 1 - p = 0.40$

a) What is the probability that non of the five vegetable servings is charred?

We can model this situation using binomial probability distribution, as our $p$ is a constant in each experiment. An experiment is repeated independently $n$ times. The probability of observing events $A$ is $p = P(A)$ and $\bar{A}$ is $q = 1 - p$.

The task condition can be reformulated:
What is the probability that we observe $\bar{A}$ (serving is not charred) 5 times if the experiment is repeated 5 times?

From the definition:

$$Z_i = \begin{cases} 1, & \text{if } A \\ 0, & \text{else} \end{cases}$$

$$X = \sum_{i=1}^{n} Z_i = \sum_{i=1}^{n} z_i = x$$ corresponds to $x$ times event $A$ and $n-x$ times event $\bar{A}$

The order is irrelevant, so the number of possibilities

$$f(x) = \begin{cases} \binom{n}{x} p^x (1-p)^{n-x}, & \text{if } x \in \{0, 1, \dots, n\} \\ 0, & \text{else} \end{cases}$$

It can be reformulated in terms of combinatorics:

$$P(k) = C(n, k) \cdot p^k \cdot q^{(n-k)},$$

where $C(n, k)$ is a combination of $n$ items taken $k$ times

If A - the portion is charred, then:

we need to find P(0) - the probability that 0 servings are charred

$n = 5$

$x = 0$ $\quad P(X=0) = \binom{5}{0} \cdot 0{,}60^0 \cdot 0{,}40^{5-0}$ or $C(5,0) \cdot 0{,}60^0 \cdot 0{,}40^5$

$p = 0.60$

$q = 0.40$

$C(5,0) = \frac{5!}{0!\,(5-0)!} = 1$

$0{,}60^0 = 1$

$0{,}40^5 = 0{,}01024$

$P(X=0) = 1 \cdot 1 \cdot 0{,}01024 = 0{,}01024$

Answer: 0,01024 or 1,024 %

b) What is the probability that exactly two of the five vegetable servings are charred?

Following the previous explanations about using binomial probability distribution:

$n = 5$ $\quad F_X(x) = P(\{a \in \Omega : X(\omega) \leq x\}), x \in \mathbb{R}$

$x = 2$ $\quad$ shortly $F(x) = P(X \leq x)$

$p = 0.60$

$q = 0.40$

$F(2) = P(X \leq 2) = \binom{5}{2} 0{,}60^2\, 0{,}40^{5-2}$ or $C(5,2)\, 0.60^2\, 0.40^3$

$C(5,2) = \frac{5!}{2!\,(5-2)!} = \frac{120}{2 \cdot 6} = 10$

$0{,}60^2 = 0{,}36$

$0{,}40^3 = 0{,}064$

$P(Y=2) = 10 \cdot 0{,}36 \cdot 0{,}064 = 10 \cdot 0{,}02304 = 0{,}2304$

Answer: 0,2304 or 23,04%

c) What is the probability that at least three out of five servings are charred?

"At least 3" means $P(3)$, $P(4)$, $P(5)$, or formally:

$$P(X \geq 3) = 1 - P(X < 3) = 1 - \underbrace{F(3-0)}$$

denotes the left-sided limit of $F(3-0) = \lim_{\varepsilon \to 0} F(3-\varepsilon)$, with $\varepsilon > 0$

The task can be solved from this two perspectives:

I

$$P(\text{at least } 3) = P(3) + P(4) + P(5)$$

$$P(3) = C(5,3) \cdot 0{,}60^3 \cdot 0{,}40^{5-3} = \frac{5!}{3!\,(5-3)!} \cdot 0{,}216 \cdot 0{,}16 = 0{,}3456$$

$$P(4) = C(5,4) \cdot 0{,}60^4 \cdot 0{,}40^{5-4} = \frac{5!}{4!\,(5-4)!} \cdot 0{,}1296 \cdot 0{,}40 = 0{,}2592$$

$$P(5) = C(5,5) \cdot 0{,}60^5 \cdot 0{,}40^{5-5} = \frac{5!}{5!\,0!} \cdot 0{,}07776 \cdot 1 = 0{,}07776$$

$$P(\text{at least } 3) = 0{,}3456 + 0{,}2592 + 0{,}07776 = 0{,}68256$$

II

Using the complement rule:

$$P(X \geq a) = 1 - F(a-0)$$

$$P(X \geq 3) = 1 - F(3-0) : \quad P(X < 3) \neq P(X \leq 3) \text{ with respect to } \varepsilon \to 0$$

$$P(X \geq 3) = 1 - P(X \leq 2)$$

The cumulative probability $P(X \leq 2)$ is the sum of $P(0)$, $P(1)$, $P(2)$

$$P(0) = C(5,0) \cdot 0{,}60^0 \cdot 0{,}40^5 = 0{,}01024$$

$$P(1) = C(5,1) \cdot 0.60^1 \cdot 0.40^4 = 0{,}0768$$

$$P(2) = C(5,2) \cdot 0.60^2 \cdot 0.40^3 = 0{,}2304$$

$$P(X \leq 2) = P(0) + P(1) + P(2) = 0{,}01024 + 0{,}0768 + 0{,}2304 = 0{,}31744$$

$$P(X \geq 3) = 1 - P(X \leq 2) = 1 - 0{,}31744 = 0{,}68256$$

Answer: 0,68256 or 68,256 %

# Task 3

> Let X be the weekly return of stock A. We assume that it follows a normal distribution with $\mu = 0.03$ and $\sigma^2 = 0.02$. We invest 40% of

> our capital in 10 such stocks and the remaining 60% in a risk-free asset with return $r_f = 0.01$.

- **i.** is larger than 0.01;
- **ii.** is larger or equal to 0.01;
- **iii.** is less than 0.01;
- **iv.** lies between 0.01 and 0.03.

> (b) Determine the distribution of the portfolio return by stating the distribution and computing its parameters. (Use the properties of the normal distribution, e.g., linear transformation).

> (a) Calculate the probability that the return of a single stock is:

> (c) How can a correlation among the assets influence the variance of the investment? Which formula supports your idea?

3. Let $X$ be the weekly return of stock A. We assume that it follows a normal distribution with $\mu = 0.03$ (3%) and $\sigma^2 = 0.02$. We invest 40% of our capital in 10 such stocks and the remaining 60% in a risk-free asset with return $r_f = 0.01$.

The weekly return of stock A is a random variable $X$ that follows a normal distribution:

$$X \sim \mathcal{N}(\mu = 0.03, \sigma^2 = 0.02)$$

$$F(x) = \int_{-\infty}^{x} f(t)\,dt \quad \text{for all } x \in \mathbb{R}$$

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left\{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right\}, \quad x \in \mathbb{R}$$

with parameter space: $\mu \in \mathbb{R}$, $\sigma > 0$

a) Calculate the probability that the return of a single stock:

i) is larger than 0.01 : $P(X > 0.01)$

$P(X > a) = 1 - P(X \leq a) = 1 - F(a)$ where $F(a)$ is the CDF of the normal distribution

Using the standard normal distribution and its properties:

$\mathcal{N}(0,1) - \Phi$ with density $\phi(y)$

Since the standard normal distribution is symmetric about zero: $\varphi(x) = \varphi(-x)$, it follows that $\Phi(x) = 1 - \Phi(-x)$

If $X \sim \mathcal{N}(\mu, \sigma^2)$, then $\frac{X-\mu}{\sigma} \sim \Phi$ and this implies:

$$F_X(x) = P\left(\frac{X-\mu}{\sigma} \leq \frac{x-\mu}{\sigma}\right) = \Phi\left(\frac{x-\mu}{\sigma}\right)$$

If $X \sim \Phi$, then $\mu + \sigma X \sim \mathcal{N}(\mu, \sigma^2)$

So that, we can standardize the normal variable $X$ to a standard normal variable $Z$:

$$Z = \frac{X-\mu}{\sigma}$$

and then apply standard normal distribution to the original variable with normal distribution

For $P(X > 0.01)$ wher $X \sim N(\mu = 0.03, \sigma^2 = 0.02)$ we can calculate standart normal variable z:

$$Z = \frac{X - \mu}{\sigma} = \frac{0.01 - 0.03}{\sqrt{0.02}} = \frac{-0.02}{0.1414} \approx -0,141421$$

with $Z \sim N(0,1)$ $P(X > 0.01) = P(Z > -0,141421)$

Standart normal distribution is symmetrie about zero:

with $z = 0,141421$ and $-z = -0,141421$ and $P(X > a) = 1 - P(X \leq a) = 1 - F(a)$

$P(Z > -z) = 1 - P(Z \leq -z)$

$P(Z \leq -z) = 1 - P(Z \leq z)$ as $\Phi(z) = 1 - \Phi(-z)$

$P(Z > -z) = 1 - 1 + P(Z \leq z) = P(Z \leq z)$

$P(Z > -0,141421) = P(Z \leq 0,141421) = \Phi(0,141421)$ using the CDF

From now three approaches can be used:

1. For simplicity we can accept $\Phi(0,141421) \sim \Phi(0.14)$ and use the table value $\approx 0.5557$ as an answer.
2. Use calculator $\approx 0.5562$
3. Estimate $\Phi(0,141421)$ using linear interpolation

$$y = y_0 + \left(\frac{x - x_0}{x_1 - x_0}\right) \cdot (y_1 - y_0)$$

$x$ - z-score we want to estimate $\Phi(z)$ $(x = 0,141421)$

$y$ - $\Phi(x)$, the cummulative probability up to $x$.

$(x_0, y_0)$ and $(x_1, y_1)$ are the known points from the standart normal tables.

Lets accept lower bound $z = x_0$ and upper bound $(z = x_1)$ as 0.14 and 0.15 respectively.

$x = 0,141421$

$x_0 = 0.14$

$x_1 = 0.15$

$y_0 = \Phi(0,14) = 0,5557$

$y_1 = \Phi(0,15) = 0.5596$

$$\Phi(0,141421) \approx 0,5557 + \frac{0,141421 - 0,14}{0.15 - 0.14}(0,5596 - 0,5557)$$

$$\approx 0,5557 + 0,000554 19 \approx$$

$$\approx 0,556254$$

Answer: 0,5562 or 55,62%

ii) is larger or equal / 0.01 : $P(X \geq 0.01)$

Using the properties of continuous distribution :

$$P(X = x) = 0 \text{ for all } x \rightarrow P(a < X < b) = P(a \leq X \leq b)$$

$$P(X \geq 0.01) = P(X > 0.01) + P(X = 0.01)$$

$0.5562$ (from previous part) $\quad 0$ (from property)

$$P(X \geq 0.01) = P(X > 0.01) = 0.5562$$

Answer : 0,5562 or 55,62 %

iii) is less than 0.01 : $P(X < 0.01)$

Using the complement rule :

$$P(X < 0.01) = 1 - P(X \geq 0.01)$$

$0.5562$ (from previous part)

$$P(X < 0.01) = 1 - 0.5562 = 0.4438$$

Alternitively, using the ideal from part i and standart normal variable Z :

$$Z = -0.141421$$

$$P(Z < -0.141421) = \Phi(-0.141421)$$

$\Phi(-z) = 1 - \Phi(z)$, so :

$$\Phi(-0,141421) = 1 - \Phi(0,141421) = 1 - 0.5562 = 0.4438$$

$0.5562$ (from previous part)

Answer: 0,4438 or 44,38 %

iv) lies between 0.01 and 0.03 : $P(0.01 < X < 0.03)$

Using the property of continuous distributions, it holds $P(X=x)=0$, so the probabilities for strict inequalities ($<$) and inclusive inequalities ($\leq$) are the same.

$P(0.01 < X < 0.03) = P(0.01 < X \leq 0.03) = F(0.03) - F(0.01)$

We can standardize the normal variable $X$ to standard normal variable $Z$:

For $X=0.01$: $Z_1 = \frac{0.01-0.03}{0.141421} = -0.141421$

For $X=0.03$: $Z_2 = \frac{0.03-0.03}{0.141421} = 0$

$P(0.01 < X < 0.03) = P(-0.141421 < Z < 0) = \underset{\text{from table}}{\underbrace{\Phi(0)}} - \underset{\text{from previous part}}{\underbrace{\Phi(-0.141421)}}$

$\Phi(0) = 0.5$

$\Phi(-0.141421) = 0.44438$

$P(0.01 < X < 0.03) = 0.5 - 0.44438 = 0.0562$

Answer: 0,0562 or 5,62 %

b) Determine the distribution of the portfolio return by stating the distribution and computing its parameters.

Portfolio Composition:
- stocks: 40% invested equally in 10 stocks
- risk-free asset: 60% invested at a return of $r_f = 0.01$

Lets determine the return distribution of the stock portion:

$R_S$ - average return of the 10 stocks

Since each stocks return $X_i$ is normally distributed $N(0.03, 0.03)$ and the stock are assumed to be independent and identically distributed, the average return $R_S$ is also normally distributed.

Using the important statements about expectation and variance: Let $X_1, \dots, X_n$ be independent with $E(X_i)=\mu$, $Var(X_i)=\sigma^2$, then sample mean $\bar{X}_n = \frac{1}{n} \cdot \sum_{i=1}^{n} X_i$ has the expectation $\mu$ and variance $\sigma^2/n$.

So that we expect $E(R_s) = \mu = 0.03$ and $Var(R_s) = \frac{\sigma^2}{n} = \frac{0.02}{10} = 0.002$.

Lets prove it:

Consider the portfolio consisting of n assets and its return R.

We denote the returns of n assets by $R_1, \dots, R_n$ and their relative fractions $w_1, \dots, w_n$. This implies $\sum_{i=1}^{n} w_i = 1$. Then the portfolio return equals $R = \sum_{i=1}^{n} w_i R_i$:

$$E(R) = E\left(\sum_{i=1}^{n} w_i R_i\right) = \sum_{i=1}^{n} E(w_i R_i) = \sum_{i=1}^{n} w_i E(R_i) = E(R_i)$$

If $E(R_i) = \mu$ for all $i = 1, \dots, n$, then $E(R) = \mu$ too

$$Var(R) = Var\left(\sum_{i=1}^{n} w_i R_i\right) = \sum_{i=1}^{n} Var(w_i R_i) = \sum_{i=1}^{n} w_i^2 Var(R_i) = w_i Var(R_i)$$

$$= \frac{1}{n} Var(R_i)$$

From the linear transformation properties:

$$E(a + bX) = a + b \cdot E(X)$$

$$Var(a + bY) = b^2 Var(X)$$

If we use scaling factor $a = \frac{1}{10}$, where 10 is an amount of stocks, and $X_i$ is the random variable that describes one investment:

$Y_i = a X_i$ - transformed variable, that is a part of $R_s$, scaled by a (can be considered as relative fraction)

$$E(Y_i) = a E(X_i) = a\mu$$

$$Var(Y_i) = a^2 Var(X_i) = a^2 \sigma^2$$

$R_s = \sum_{i=1}^{10} Y_i$ : $E(R_s) = \sum_{i=1}^{10} E(Y_i)$ and $Var(R_s) = \sum_{i=1}^{10} Var(Y_i)$ since all $Y_i$ are independent and similarly distributed

$$E(R_s) = E\left(\sum_{i=1}^{10} (Y_i)\right) = E\left(\sum_{i=1}^{10} \left(\frac{1}{10} X_i\right)\right) = E\left(\frac{1}{10} \sum_{i=1}^{10} X_i\right) = \frac{1}{10} \sum_{i=1}^{10} E(X_i) = \frac{1}{10} \cdot 10\mu = \mu = 0.03$$

$$Var(R_s) = Var\left(\sum_{i=1}^{n} (Y_i)\right) = Var\left(\sum_{i=1}^{10} \left(\frac{1}{10} X_i\right)\right) = Var\left(\frac{1}{10} \sum_{i=1}^{10} X_i\right) =$$

$$= \left(\frac{1}{10}\right)^2 \sum_{i=1}^{10} Var(X_i) = \left(\frac{1}{10}\right)^2 \cdot 10 \cdot \sigma^2 = \frac{\sigma^2}{10} = \frac{0.02}{10} = 0.002$$

$$R_s \sim \mathcal{N}(\mu_s = 0.03, \sigma_s^2 = 0.002)$$

Lets determine the full portfolio return $R_p$:

$$R_p = w_s R_s + w_f r_f, \text{ where}$$

$w_s = 0.40$ (weight in stocks)
$w_f = 0.60$ (weight in risk-free asset)
$r_f = 0.01$ (return of risk-free asset)

Since $r_f$ is a constant: $E(r_f) = r_f = 0.01$
$Var(r_f) = 0$
$Cov(R_s, r_f) = 0$

Using the linearity of expectation and considering $R_s$ and $r_f$ as two component of multivariate RV:

$$E(R_p) = E(w_s R_s + w_f r_f) = E(w_s R_s) + E(w_f r_f) =$$
$$= w_s E(R_s) + w_f E(r_f) = 0.40 \cdot 0.03 + 0.60 \cdot 0.01 = 0.018$$

Using the linearity rules for variance and rules for covariance:

$$Var(aX + bY) = a^2 Var(X) + b^2 Var(Y) + 2ab\, Cov(X,Y), \text{ since}$$
$$Var(aX + bY) = E[(aX + bY - E(aX + bY))^2] =$$
$$= E[a(X - E(X)) + b(Y - E(Y))]^2 =$$
$$= a^2 Var(X) + b^2 Var(Y) + 2ab\, Cov(X,Y)$$

$$Var(R_p) = Var(w_s \cdot R_s + w_f \cdot r_f) =$$
$$= w_s^2 Var(R_s) + w_f^2 \cdot \underbrace{Var(r_f)}_{=0} + 2 w_s w_f \underbrace{Cov(R_s, r_f)}_{=0} =$$
$$= w_s^2 Var(R_s) = 0.40^2 \cdot 0.002 = 0.00032$$

Standard deviation can be calculated as:

$$\sigma_p = \sqrt{Var(R_p)} = \sqrt{0.00032} \approx 0.01789$$
$$R_p \sim \mathcal{N}(\mu_p = 0.018, \sigma_p^2 = 0.00032)$$

Answer: $\mu_p = 0.018$ or 1.8%
$\sigma_p^2 = 0.00032$
$\sigma_p = 0.01789$ or 1.789%

$R_p \sim \mathcal{N}(0.018, 0.00032)$

c) How a correlation among the assets can influence the variance of the investment?

When assets in portfolio are correlated, it affects the overall variance (risk) of the portfolio:

- Positive correlation ($\rho > 0$): increases portfolio variance
- Negative correlation ($\rho < 0$): decreases portfolio variance

Since the covariance is a measure of the joint variability of two random variables, the sign of the covariance shows the tendency in the linear relationship between the variables.

- Covariance is positive, if greater (smaller) values of one variable mainly correspond with greater (smaller) values of the other variable.
- Covariance is negative, if greater values of one variable mainly correspond to lesser values of the other.

The correlation coefficient normalizes the covariance by dividing by the geometric mean of the total variances for the two random variables and the correlation itself expresses the extent to which two variables are linearly related.

For a portfolio of $n$ assets with equal weight and identical variances, the variance of the average return $R_S$ is:

$$Var(R_S) = \frac{\sigma^2}{n}[1 + (n-1)\rho], \text{ where}$$

$\sigma^2$ is the variance of individual assets
$\rho$ is the average correlation coefficient
$n$ is the number of assets

without correlation ($\rho = 0$): $Var(R_S) = \frac{\sigma^2}{n}$, variance decreases with more assets due to diversification.

with positive correlation ($\rho > 0$): the term $(n-1)\rho$ increases the overall variance and reduces the benefits of diversification.

with negative correlation ($\rho < 0$): the term $1 - (n-1)\rho$ decreases the overall variance

For a portfolio consisting of n assets, the total return $R_p$ is a weighted sum of the individual asset returns:

$$R_p = \sum_{i=1}^{n} w_i R_i \text{ , where}$$

$R_i$ is the return of asset $i$

$w_i$ is the weight allocated to asset $i$

$$\sum_{i=1}^{n} w_i = 1$$

The variance of the portfolio return $R_p$ is:

$$Var(R_p) = Var\left(\sum_{i=1}^{n} w_i R_i\right)$$

For any set of random variables, the variance of the weighted sum is:

$$Var\left(\sum_{i=1}^{n} w_i R_i\right) = \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j Cov(R_i, R_j)$$

The covariance between two assets $i$ and $j$ is defined as:

$$Cov(R_i, R_j) = \rho_{ij} \sigma_i \sigma_j \text{ , where}$$

$\rho_{ij}$ is the correlation coefficient

$\sigma_i, \sigma_j$ are the standard deviations of the returns

By substituting the covariance expression:

$$Var(R_p) = \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j \rho_{ij} \sigma_i \sigma_j$$

When $i=j$, $\rho_{ij} = 1$ (an asset is perfectly correlated with itself):

$$w_i w_j \rho_{ij} \sigma_i \sigma_j = w_i^2 \sigma_i^2$$

these variance terms represent the contribution of each asset's individual variance to the portfolio variance.

When $i \neq j$: $w_i w_j \rho_{ij} \sigma_i \sigma_j$

these covariance terms represent the contribution of the covariance of different assets to the portfolio variance (captures how the returns of assets move together)

$\rho_{ij} > 0$: increases the variance, because assets tend to move at the same direction, amplifying overall portfolio risk.

$\rho_{ij} < 0$: decreases the variance, because assets tend to move in opposite directions, offsetting each other's variability and reducing the risk.

$\rho_{ij} = 0$ : covariance terms dropout, and only individual variances contribute to the portfolio variance, because assets move independently of each other, providing the benefits of diversification.

Suppose all assets have the same variance $\sigma^2$ and the same correlation $\rho$ between each other (each pair):

- weight are equal $w_i = \frac{1}{n}$
- $\rho_{ij} = \rho$ for all $i \neq j$
- $\rho_{ij} = 1$ for all $i = j$

The variance formula simplifies to:

$$Var(R_p) = \sum_{i=1}^{n} w_i^2 \sigma_i^2 + \sum_{i=1}^{n} \sum_{\substack{j=1 \\ j \neq i}}^{n} w_i w_j \rho \sigma^2$$

Since $w_i = \frac{1}{n}$ :

individual variance: $\sum_{i=1}^{n} w_i^2 \sigma_i^2 = \sum_{i=1}^{n} \left(\frac{1}{n}\right)^2 \sigma^2 = n \frac{1}{n^2} \sigma^2 = \frac{\sigma^2}{n}$

covariances: $\sum_{i=1}^{n} \sum_{\substack{j=1 \\ j \neq i}}^{n} w_i w_j \rho \sigma^2 = \sum_{i=1}^{n} \sum_{\substack{j=1 \\ j \neq i}}^{n} \left(\frac{1}{n}\right)\left(\frac{1}{n}\right) \rho \sigma^2 =$

$$= n \cdot (n-1) \frac{1}{n^2} \rho \sigma^2 = \frac{(n-1)}{n} \rho \sigma^2$$

Total portfolio variance:

$$Var(R_p) = \frac{\sigma^2}{n} + \frac{(n-1)}{n} \rho \sigma^2 = \frac{\sigma^2}{n} [1 + (n-1)\rho]$$

Supporting formula summarized:

$$Var(R_p) = \sum_{i=j}^{n} \sum_{i \neq j}^{n} w_i w_j \, Cov(R_i, R_j) = \frac{\sigma^2}{n} [1 + (n-1)\rho]$$

From our example:

variance of each asset: $\sigma^2 = 0.02$

number of assets: $n = 10$

example of correlation: $\rho = 0.5$

$Var(R_p) = \frac{0.02}{10} [1 + (10-1) \cdot 0.5] = 0.011$

is higher (0.009 more) than the uncorrelated case

Answer: positive correlation reduces the risk reduction typically gained from diversification.

# Task 4

> Download (for example, from Yahoo Finance) the weekly prices and calculate the returns of Tesla Inc for the last two years (use simple

> returns $P_t$ - $P_{t-1}$ )/$P_{t-1}$ or continuous returns ln( $P_t$/$P_{t-1}$ )). Multiply the returns by 51 to obtain annualized returns. You are interested in the probability that the Tesla return falls:

- a) below -2%;
- b) below -4%.

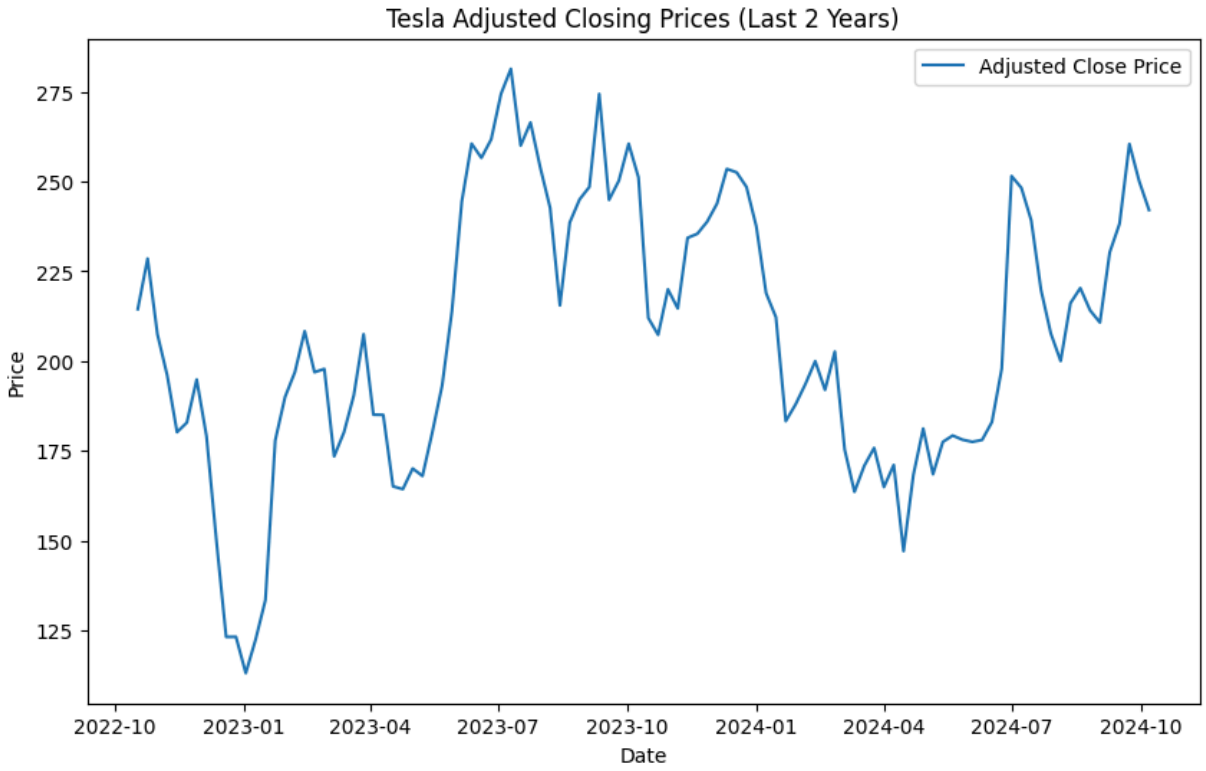
In [2]:
```
# Download Tesla stock data for the last two years (weekly)
tesla = yf.download('TSLA', period='2y', interval='1wk')

# Preview the data
tesla.head()
```

```
[**********100%**********]  1 of 1 completed
```

Out[2]:

| Date | Open | High | Low | Close | Adj Close | Volume |
|---|---|---|---|---|---|---|
| **2022-10-10** | 223.929993 | 226.990005 | 204.160004 | 204.990005 | 204.990005 | 397406400 |
| **2022-10-17** | 210.039993 | 229.820007 | 202.000000 | 214.440002 | 214.440002 | 415404100 |
| **2022-10-24** | 205.820007 | 233.809998 | 198.589996 | 228.520004 | 228.520004 | 412758400 |
| **2022-10-31** | 226.190002 | 237.399994 | 203.080002 | 207.470001 | 207.470001 | 342474400 |
| **2022-11-07** | 208.649994 | 208.899994 | 177.119995 | 195.970001 | 195.970001 | 596889200 |

In [11]:
```
# Plot the adjusted closing prices
plt.figure(figsize=(10, 6))
plt.plot(tesla.index, tesla['Adj Close'], label='Adjusted Close Price')
plt.title('Tesla Adjusted Closing Prices (Last 2 Years)')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.show()
```




In [3]:
```
# Calculate simple returns
tesla['Simple Returns'] = tesla['Adj Close'].pct_change()

# Calculate continuous returns
tesla['Log Returns'] = np.log(tesla['Adj Close'] / tesla['Adj Close'].shift(

# Remove the first row with NaN
tesla = tesla.dropna()

# Preview the returns
tesla[['Simple Returns', 'Log Returns']].head()
```

Out[3]:

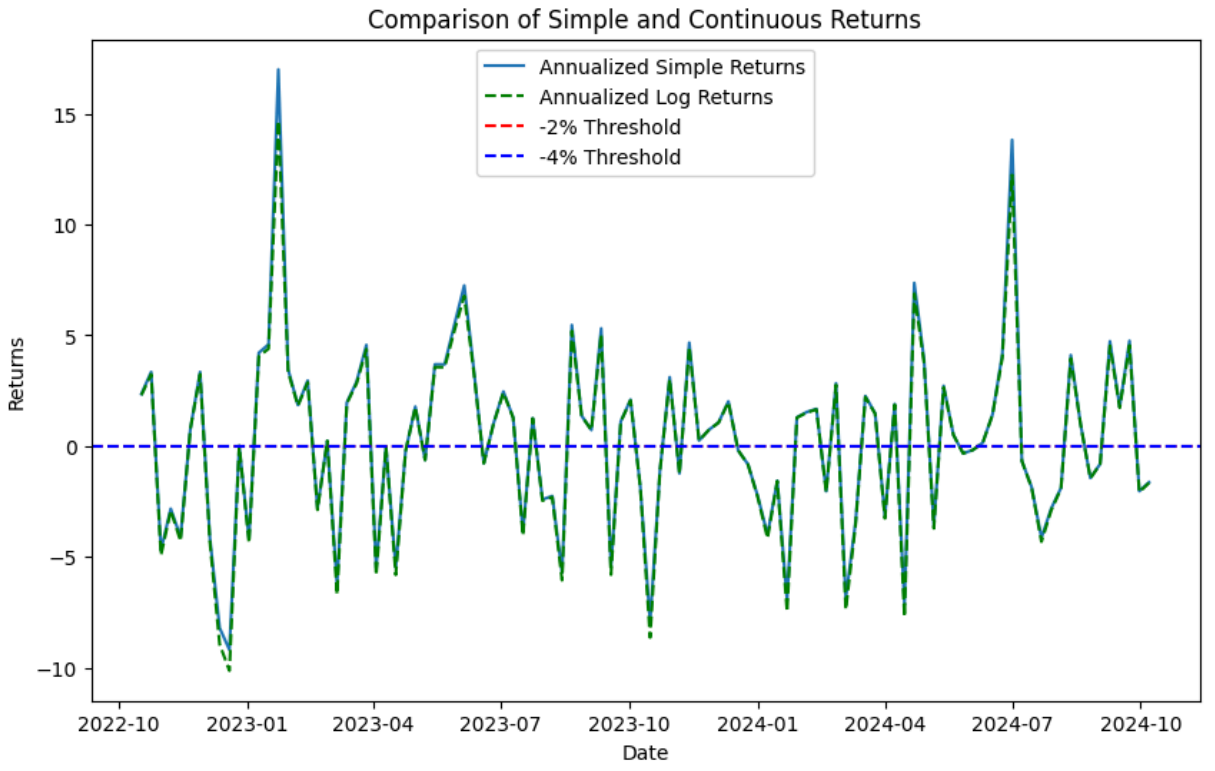
| Date | Simple Returns | Log Returns |
|---|---|---|
| **2022-10-17** | 0.046100 | 0.045069 |
| **2022-10-24** | 0.065659 | 0.063594 |
| **2022-10-31** | -0.092114 | -0.096637 |
| **2022-11-07** | -0.055430 | -0.057025 |
| **2022-11-14** | -0.080523 | -0.083950 |

In [5]:
```
# Annualize returns
tesla.loc[:, 'Annualized Simple Returns'] = tesla['Simple Returns'] * 51
tesla.loc[:, 'Annualized Log Returns'] = tesla['Log Returns'] * 51

# Preview annualized returns
tesla[['Annualized Simple Returns', 'Annualized Log Returns']].head()
```

Out[5]:

| Date | Annualized Simple Returns | Annualized Log Returns |
|---|---|---|
| **2022-10-17** | 2.351089 | 2.298507 |
| **2022-10-24** | 3.348629 | 3.243282 |
| **2022-10-31** | -4.697839 | -4.928487 |
| **2022-11-07** | -2.826915 | -2.908283 |
| **2022-11-14** | -4.106649 | -4.281436 |

In [20]:
```
# Plot both simple and continuous returns for comparison
plt.figure(figsize=(10, 6))
plt.plot(tesla.index, tesla['Annualized Simple Returns'], label='Annualized
plt.plot(tesla.index, tesla['Annualized Log Returns'], linestyle='dashed', l
plt.axhline(y=-0.02, color='r', linestyle='--', label='-2% Threshold')
plt.axhline(y=-0.04, color='b', linestyle='--', label='-4% Threshold')
plt.title('Comparison of Simple and Continuous Returns')
plt.xlabel('Date')
plt.ylabel('Returns')
plt.legend()
plt.show()
```




1. **Tesla Adjusted Closing Prices**:

   - The plot shows the adjusted closing prices of Tesla Inc. over the last two years. The price exhibits significant fluctuations, reflecting high volatility in Tesla's stock performance during this period.

2. **Comparison of Simple and Continuous Returns**:

   - The second plot compares the annualized simple and log returns for Tesla. Both types of returns closely align with each other over time.
   - The thresholds of -2% and -4% are marked, showing how frequently the returns drop below these levels.

# Part A

> Calculate this probability using the tools and methods of the descriptive statistics (quantiles, empirical CDF, etc.)

In [6]:
```
# Calculate the empirical probabilities using the annualized simple returns
empirical_prob_a = (tesla['Annualized Simple Returns'] < -0.02).mean()
empirical_prob_b = (tesla['Annualized Simple Returns'] < -0.04).mean()

print(f"Empirical probability that the return falls below -2%: {empirical_pr
print(f"Empirical probability that the return falls below -4%: {empirical_pr
```

```
Empirical probability that the return falls below -2%: 0.4423
Empirical probability that the return falls below -4%: 0.4423
```

In [13]:
```
# 95% Value at Risk (VaR)
VaR_95 = np.percentile(tesla['Annualized Simple Returns'], 5)

# 99% Value at Risk (VaR)
VaR_99 = np.percentile(tesla['Annualized Simple Returns'], 1)

print(f"95% Value at Risk (VaR): {VaR_95:.4f}")
print(f"99% Value at Risk (VaR): {VaR_99:.4f}")
```

```
95% Value at Risk (VaR): -6.7820
99% Value at Risk (VaR): -8.2011
```

1. **Empirical Probability of Negative Returns**:

   - The empirical probability that Tesla's return falls below -2% is approximately 44.23%. This indicates that in almost half of the observed periods, Tesla's returns dipped below this threshold.
   - Similarly, the probability that the return falls below -4% is also 44.23%, suggesting that sharp drops in returns are not uncommon for Tesla.

2. **Value at Risk (VaR)**:

   - The 95% Value at Risk (VaR) is -6.78%, meaning that in the worst 5% of cases, Tesla's returns could fall by at least 6.78%.
   - The 99% VaR is -8.20%, implying that in the worst 1% of scenarios, returns could drop by at least 8.20%. This highlights the potential risk associated with Tesla's stock in extreme market conditions.

# Part B

> Calculate this probability assuming that the returns follow a normal distribution with the parameters given by the sample mean and sample variance.

In [8]:
```
# Calculate sample mean and variance
mean = tesla['Annualized Simple Returns'].mean()
std = tesla['Annualized Simple Returns'].std()

# Calculate probabilities assuming normal distribution
prob_a_normal = norm.cdf(-0.02, loc=mean, scale=std)
prob_b_normal = norm.cdf(-0.04, loc=mean, scale=std)

print(f"Probability (normal dist) that the return falls below -2%: {prob_a_r
print(f"Probability (normal dist) that the return falls below -4%: {prob_b_r
```

```
Probability (normal dist) that the return falls below -2%: 0.4745
Probability (normal dist) that the return falls below -4%: 0.4726
```

**Probability Based on Normal Distribution**:

- The probability that Tesla's return falls below -2%, assuming a normal distribution, is 47.45%. This is slightly higher than the empirical probability, indicating a smoother distribution of returns compared to the actual observed data.
- The probability of returns falling below -4% under a normal distribution is 47.26%, also slightly higher than the empirical probability. This suggests that the normal distribution may overestimate the likelihood of larger negative returns compared to the real data.

# Part C

> Compare the probabilities. Discuss pros and cons of both methods.

## Comparison of Probabilities and Discussion

1. **Empirical vs. Normal Distribution Probabilities**:
   - **Empirical Probability**: The empirical probabilities for returns falling below -2% and -4% are both 44.23%. These values are based on the actual observed data and reflect the real-world behavior of Tesla's stock returns during the period.
   - **Normal Distribution Probability**: The probabilities assuming a normal distribution are slightly higher, with 47.45% for returns below -2% and

47.26% for returns below -4%. This suggests that the normal distribution smooths out the return behavior, potentially overestimating the likelihood of more extreme returns.

2. **Pros and Cons**:
    - **Empirical Method**:
        - **Pros**: Provides a realistic assessment based on actual historical data, accounting for any irregularities or non-normality in the data (e.g., skewness, kurtosis, fat tails).
        - **Cons**: It may not generalize well to future events, especially in the presence of limited historical data or significant market changes. It is sensitive to the particular dataset used, and outliers can heavily influence the results.
    - **Normal Distribution Method**:
        - **Pros**: Provides a theoretically sound, smooth estimation that can be applied even with limited data. It assumes returns follow a bell curve, which simplifies risk estimation and can be useful in many financial models (e.g., for VaR calculations).
        - **Cons**: The assumption of normality might not hold in practice, especially for volatile stocks like Tesla. Real returns often exhibit fat tails, making extreme events more likely than the normal distribution predicts. This method can underestimate the risk of extreme losses.
3. **Conclusion**:
    - While the normal distribution method offers simplicity and smoothness, it may not accurately reflect the true risk of extreme returns for volatile stocks. The empirical method, though more sensitive to the dataset, provides a more accurate reflection of past performance and potential risks. Ideally, both methods should be used together, with adjustments made based on the stock's characteristics and the market conditions.

# Task 5

> Assume that the price of a given asset can either decrease or increase in every period of time. This setting is called a binomial model. The probability of an upward movement equals 55%, and the starting price S(0) = 10 . The upward and downward returns equal 0.02 and -0.01 respectively. Consider holding the asset for several periods.

# Part A

> Calculate the probability that the stock price increases:

- **i.** Five periods in a row.
- **ii.** Three periods in a row.
- **iii.** In three out of five periods.
- **iv.** In 10 out of 15 periods.

In [22]:
```
# Given values
p_up = 0.55  # probability of upward movement
p_down = 1 - p_up  # probability of downward movement

# i. Probability of increasing 5 periods in a row
prob_5_up_in_a_row = p_up ** 5

# ii. Probability of increasing 3 periods in a row
prob_3_up_in_a_row = p_up ** 3

# iii. Probability of increasing in 3 out of 5 periods (binomial distributio
prob_3_out_of_5 = stats.binom.pmf(3, 5, p_up)

# iv. Probability of increasing in 10 out of 15 periods
prob_10_out_of_15 = stats.binom.pmf(10, 15, p_up)

print(f"Probability that the stock price increases for 5 periods in a row: {
print(f"Probability that the stock price increases for 3 periods in a row: {
print(f"Probability that the stock price increases in 3 out of 5 periods: {p
print(f"Probability that the stock price increases in 10 out of 15 periods:
```

```
Probability that the stock price increases for 5 periods in a row: 0.0503
Probability that the stock price increases for 3 periods in a row: 0.1664
Probability that the stock price increases in 3 out of 5 periods: 0.3369
Probability that the stock price increases in 10 out of 15 periods: 0.1404
```

# Part B

> Calculate the expected return of this investment after two periods.

In [23]:
```
upward_return = 0.02
downward_return = -0.01

# Expected return after two periods
expected_return_2_periods = (
    (p_up * upward_return + p_down * downward_return) ** 2
)

print(f"Expected return after 2 periods: {expected_return_2_periods:.4e}")
```

```
Expected return after 2 periods: 4.2250e-05
```

## Probabilities:

- **i.** The probability that the stock price increases for five periods in a row is **5.03%**. This shows a low likelihood of continuous upward movement for five consecutive periods.
- **ii.** The probability that the stock price increases for three periods in a row is **16.64%**, indicating a higher but still moderate chance of continuous upward movement.
- **iii.** The probability that the stock price increases in three out of five periods is **33.69%**, suggesting a more reasonable chance that the stock will move upward in the majority of the periods.
- **iv.** The probability that the stock price increases in 10 out of 15 periods is **14.04%**, which is relatively lower, showing that it is less likely for the stock to rise in two-thirds of the periods.

## Expected Return:

- The expected return after two periods is **0.00423%**, indicating a slight expected increase in the value of the investment. This is a minimal gain, which reflects the modest upward movement probabilities and the small return values for each period.

# Task 6

> Compute the following probabilities for the distributions N(0,1), $t_2$, Exp(1), $chi^2_4$:

> (a) ( P(1 < X < 1.5) )

> (b) ( P(1 ≤ X < 1.5) )

> (c) ( P(1 < X) )

> (d) ( P(X < 1.5) )

> (e) ( P(X = 1.5) )

6) Compute the following probabilities for the distributions:
$N(0,1)$, $t_2$, $Exp(1)$, $\chi^2_4$:

a) $P(1 < X < 1.5)$

b) $P(1 \leq X < 1.5)$

c) $P(1 < X)$

d) $P(X < 1.5)$

e) $P(X = 1.5)$

1. Standard Normal Distribution $N(0,1)$
Lets use CDF $\Phi(x)$ and standard normal tables to find probabilities.

a) $P(1 < X < 1.5) = \Phi(1.5) - \Phi(1) = 0.9332 - 0.8413 = 0.0919$

b) $P(1 \leq X < 1.5)$ : for continuos distributions $P(X=1)=0$, so:
$P(1 \leq X \leq 1.5) = P(1 < X < 1.5) = 0.0919$

c) $P(1 < X) = 1 - \Phi(1) = 1 - 0.8413 = 0.1587$

d) $P(X < 1.5) = \Phi(1.5) = 0.9332$

e) $P(X = 1.5)$ : for continuous distributions $P(X=1.5) = 0$

2. Students t-distribution with 2 degrees of freedom $t_2$
Lets use t-distribution tables to find probabilities.

a) $P(1 < X < 1.5) = F(1.5) - F(1) = 0.8524 - 0.7887 = 0.0637$

b) $P(1 \leq X < 1.5) = P(1 < X < 1.5) = 0.0637$

c) $P(1 < X) = 1 - F(1) = 1 - 0.7887 = 0.2113$

d) $P(X < 1.5) = F(1.5) = 0.8524$

e) $P(X = 1.5) = 0$

3. Exponential Distribution $Exp(1)$
Standard density function: $f(x) = \lambda e^{-\lambda x}$, $x \geq 0$, $\lambda > 0$
The exponential distribution with rate $\lambda = 1$ has:
- Probability density function (PDF): $f(x) = e^{-x}$ for $x \geq 0$
- Cumulative distribution function (CDF): $F(x) = 1 - e^{-x}$

a) $P(1 < X < 1.5) = F(1.5) - F(1) = (1 - e^{-1.5}) - (1 - e^{-1}) = e^{-1} - e^{-1.5}$

$e^{-1} = \frac{1}{e} \approx 0.3679$

$e^{-1.5} = e^{-1} \cdot e^{-0.5} \approx 0.3679 \cdot 0.6065 \approx 0.2231$

$P(1 < X < 1.5) = 0.3679 - 0.2231 = 0.1448$

b) $P(1 \leq X < 1.5) = P(1 < X < 1.5) = 0.1448$

c) $P(1 < X) = 1 - F(1) = e^{-1} = 0.3679$

d) $P(Y < 1.5) = F(1.5) = 1 - e^{-1.5} = 1 - 0.2231 = 0.7769$

e) $P(Y = 1.5) = 0$

4. Chi-squared Distribution with 4 degrees of freedom $\chi_4^2$

this is a special case of gamma distribution, lets use chi-squared distribution tables to find probabilities

a) $P(1 < X < 1.5) = F(1.5) - F(1) = 0.1511 - 0.0902 = 0.0639$

b) $P(1 \leq X < 1.5) = P(1 < X \leq 1.5) = 0.0639$

c) $P(1 < X) = 1 - F(1) = 1 - 0.0902 = 0.9098$

d) $P(X < 1.5) = F(1.5) = 0.1511$

e) $P(Y = 1.5) = 0$